e-Informatica Software Engineering Journal
Volume 00, Issue 0, pages: 1–39

BibTeX

Constraint Annotation Patterns for Prototype Generation within Domain-Driven Design

Van-Vinh Le, Duc-Hanh Dang*
*Corresponding author: hanhdd@vnu.edu.vn

Article info

Keywords:
Domain-Driven Design
Domain-Specific Language
UML/OCL
Annotation-Based Mechanism
Pattern.

Submitted: 1 Dec. 2025
Revised: 1 Jan 1970
Accepted: 1 Jan 1970
Available online: 1 Jan 1970

Abstract

Context: In Domain-Driven Design (DDD), domain models capture business knowledge, and OCL constraints precisely specify business rules. Current DDD methods employ annotation-based Domain-Specific Languages (aDSLs) to encode the domain model within the implementation, using annotations to specify its structure and logic. However, such aDSLs often fall short in capturing and integrating complex OCL constraints.
Objective: This paper aims to improve the expressiveness of aDSLs for specifying complex domain constraints. We introduce Constraint Annotation Patterns (CAPs), a reusable pattern abstraction that associates each domain constraint with a corresponding specification pattern, enabling constraints to be expressed and managed at the domain-model level.
Method: We define a catalog of CAPs, where each CAP is a parameterized template that captures a reusable OCL constraint structure. Concrete constraints are instantiated by binding template parameters via annotations in Domain Class Specification Language (DCSL) models. From these CAP instantiations, the corresponding OCL invariants are reconstructed and composed into unified domain models for validation and subsequent model-driven processing.
Results: Evaluation across three heterogeneous case studies (CourseMan, OrderMan, and ProcessMan) demonstrates that CAP achieves full coverage of predefined expressiveness criteria while significantly reducing manual validation code and auxiliary artifacts compared to existing annotation-based approaches.
Conclusions: The approach bridges the gap between the problem domain and the technical space while preserving the expressiveness of DDD, enabling automatic enforcement, reuse, and seamless model-to-code transformation.

1. Introduction

Domain-Driven Design (DDD) [1] is an approach to developing complex software systems that places the primary focus on the domain and its underlying logic. The core of DDD lies in the construction of a domain model that serves as the foundation for business knowledge and provides architectural guidance, with OCL constraints [2] ensuring the precise specification of business rules. The domain model guarantees *feasibility*, enabling the seamless transition between the model and the code, so that the model can be executed,

© 0000 The Authors. Published by Wrocław University of Science and Technology Publishing House.
This is an open access article under the CC BY license international.

refined, and evolved alongside the implementation, thus supporting applicability across various business contexts. It also guarantees *satisfiability* by capturing and fulfilling all business requirements and critical constraints defined by stakeholders, expressed through a ubiquitous language consistently used by domain experts and developers throughout analysis, design, and implementation.

The domain models [3–7] are usually represented in one of the following ways. First, UML/OCL class diagrams are used to capture domain concepts and their relationships [8]. Although they can be translated into executable models or object-oriented programs, a semantic gap remains between model representations and programming languages. Second, external DSLs [9–12] introduce custom syntaxes that enable precise specification of business logic, thereby facilitating constraint definition, automation, and effective communication between domain experts and developers. They are typically defined through metamodelling, which provides an explicit abstract syntax and supports validation via OCL. However, external DSLs require substantial effort to develop and maintain executable toolchains, including parsers, editors, and code generators, which may limit their adoption in agile DDD contexts. Recent DSLs, such as B-OCL [13], offer capabilities for structural modeling and limited constraint support, yet challenges remain in translating UML/OCL to executable code, integrating runtime OCL validation, and preserving lifecycle correctness. Finally, internal DSLs [14–16], especially annotation-based DSLs, embed domain logic within host languages like Java [17], enabling automatic test generation and verifiable implementations. Existing annotation-based approaches [4, 5, 18, 19] address only subsets of OCL, leaving comprehensive representation of complex constraints and fully automated code generation unresolved. Their main limitation lies in satisfiability, as host-language syntax restricts the natural representation of complex domain concepts.

Our previous works, DCSL [20] and AGL [21], employ an annotation-based DDD approach (aDSL) to construct specific domain models—both structurally and behaviorally—with the aim of supporting executable domain models for particular problem domains. These models provide concise and understandable representations of domain concepts, constraints, and business rules that closely resemble programming language syntax. Annotations are integrated directly into the source code, establishing a strong connection between domain model design and implementation. This method bridges the gap between the problem domain and the technical space by enabling the automatic generation of software artifacts directly from the domain model. Aligning domain concepts with their technical implementations reduces errors from manual programming and accelerates development.

However, these approaches remain limited, as aDSLs often fall short in capturing and integrating complex OCL constraints, which are essential for the precise specification of business rules in DDD [1, 2], and in specifying domain concerns and composing them within the executable unified domain model for automatic prototype generation, as introduced in UDML [22].

In this paper, we propose an extension of annotation-based mechanisms and specification patterns for aDSLs to express complex OCL constraints in domain models within the DDD paradigm. Each OCL constraint is mapped to a corresponding specification pattern, referred to as a Constraint Annotation Pattern (CAP), which employs annotation structures to represent constraint semantics and enables verification directly on the domain model. Furthermore, our approach supports the composition of aDSLs within an executable unified domain model, which serves as the foundation for automatic prototype generation, ensuring that both constraints and concerns are seamlessly integrated into software artifacts. By leveraging reusable CAP and concern composition mechanisms, the approach bridges the gap

between the problem domain and the technical space, facilitating the automatic generation of software artifacts while preserving the expressiveness of DDD. The methodology is evaluated through detailed case studies, focusing on expressiveness, required coding effort, and applicability in real-world DDD projects.

Our methodology makes three key contributions:

1. Extension of aDSL/DCSL with a mechanism to represent complex OCL constraints, where each constraint is linked to a corresponding CAP.
2. Definition of a catalog of CAPs, in which each CAP encapsulates: (i) a UML structural perspective, (ii) an OCL specification, and (iii) annotations embedding OCL semantics into models, enabling automatic enforcement, reuse, and seamless model-to-code transformation.
3. Development of the CAP/UDML tool as an extension of USE, enabling CAP management, annotation-based constraint instantiation, and automatic OCL reconstruction. The toolchain integrates with JDA for automated artifact generation. Evaluation on CourseMan, OrderMan, and ProcessMan demonstrates the expressiveness and effectiveness of the proposed approach.

The rest of the paper is organized as follows: Section 2 presents our motivating example and the technical background. Section 3 overviews our approach. Section 4 presents our method of constraint annotation patterns. Section 5 presents the tool support for our method.Section 6 evaluates and discusses the issues surrounding the method. Section 7 discusses threats to the validity of our work. Section 8 reviews the related work. The paper concludes with a summary of the findings and future directions.

2. Motivating Example and Background

This section motivates our work through an example and provides the necessary background. We first introduce a motivating domain model, then briefly review the principles of Domain-Driven Design (DDD), and finally present the Domain Class Specification Language (DCSL), which forms the technical foundation of our approach.

2.1. Motivating Example

Our study focuses on a course management system (CourseMan), in which the problem domain is captured by the domain model shown in Fig. 1. The domain concepts of this domain correspond to the classes: The class `Student` represents individuals enrolled at the university. Each student may register for multiple `CourseOfferings`, which represent specific instances of a `CourseModule` delivered in a given `AcademicTerm`. A `CourseModule` defines the syllabus and learning objectives of a course, whereas a `CourseOffering` associates the module with an academic term and an `Instructor` responsible for teaching it. The `Program` class represents a degree program corresponding to an official academic structure that students enroll in and follow throughout their studies. A student's participation in a program is captured by `ProgramEnrolment`, while their enrolment in specific course offerings is managed through `Enrolment`, forming a many-to-many association between `Student` and `CourseOffering`. The `TermRecord` class captures each student's academic performance for a given term, summarizing enrolments, grades, and GPA calculations.

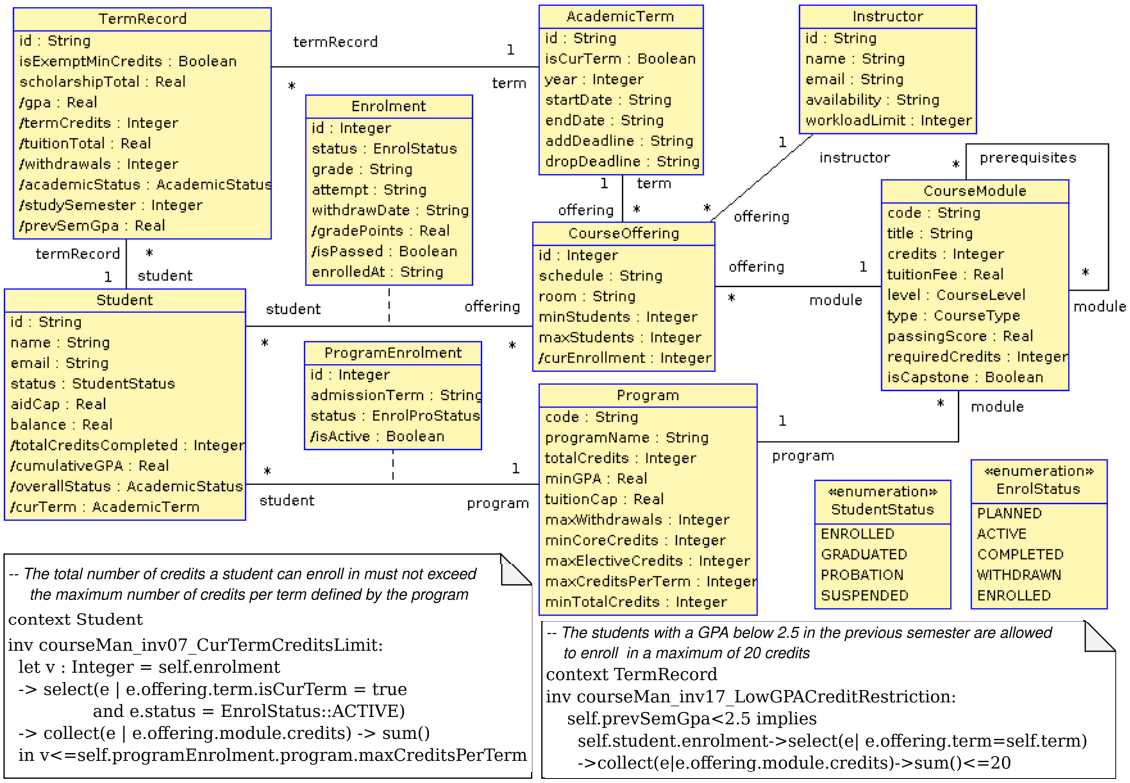

Figure 1: The **CourseMan** domain model represented as a UML/OCL class diagram.

The UML class diagram in Fig. 1 captures the structural aspects of the CourseMan domain model. To specify behavioral and logical aspects that cannot be adequately captured using UML alone, we employ OCL constraints. One such constraint, illustrated in Fig. 1, ensures that the total number of credits a student enrolls in does not exceed the maximum per term specified by the student's program.

We aim to incorporate such a UML/OCL domain model into the context of DDD. This presents a challenge, as DDD encompasses not only model-centric aspects such as UML/OCL, but also code-centric and behavior-driven approaches. A domain model in DDD, as further explained in Subsection 2.2, must be technically feasible—that is, the model should be executable or interpretable as a program with well-defined operational semantics. Consequently, stakeholders, including both customers and developers, can be provided with appropriate views that are projected from the domain model.

A promising approach to this challenge, introduced in our previous work [20], is to define an annotation-based domain-specific language, as further explained in Subsection 2.3, for specifying domain models. The core idea is to use annotations to re-express OCL constraints. However, the current approach is limited to handling only essential constraints within domain models. To be more broadly applicable, it must also support more complex OCL constraints. Addressing this limitation serves as a key motivation for the present work.

2.2. Brief Overview of Domain-Driven Design

Evans' domain-driven design (DDD) methodology addresses the challenges of developing complex software systems by placing the domain at the center and providing both strategic and tactical means for managing complexity. DDD emphasizes building software models that closely reflect actual business logic, thereby ensuring maintainability and extensibility. Approaches that automatically generate domain models from high-level specifications—such as UML/OCL diagrams—enhance domain model representation and support the automation of the software development process, including source code and artifact generation [23, 24].

Building on this foundation, several studies [3–5] apply DDD to real-world business domains, aiming to accurately reflect business requirements while maintaining technical feasibility. Additionally, research [6, 7] focuses on constructing rich domain models that encapsulate business logic and address complex requirements, such as implementing business rules, workflows, and constraints in a DDD-compliant manner, while supporting business logic validation.

Three key features of DDD include: (1) feasibility—the domain model should be executable as code and vice versa, ensuring practical applicability across diverse business contexts; (2) satisfiability—the domain model must fulfill all business requirements and critical constraints as defined by stakeholders and expressed through a ubiquitous language [1]; and (3) effectiveness—the domain model should accurately describe current business requirements to ensure effective operation while supporting future expansion and maintainability. This ubiquitous language, shared among stakeholders including domain experts, designers, and developers, is developed iteratively through agile processes of domain requirement elicitation. Enhancing domain model representation and automating the software development process to satisfy these key DDD principles remains a central focus of current research.

2.3. DCSL: An Annotation-Based DSL to Specify Domain Models

Several studies have explored the integration of OCL with annotations to represent domain models in Domain-Driven Design (DDD), particularly to enable automated validation and test generation [12, 14–16]. In parallel, domain-specific languages (DSLs) have been widely adopted to model business domains through focused and semantically rich abstractions [9–11, 25]. These studies demonstrate that DSLs can effectively capture domain knowledge while supporting automation.

In our previous work [20], we introduced DCSL (Domain Class Specification Language), an annotation-based internal DSL situated in a technical horizontal domain. DCSL provides modeling constructs for specifying the structural aspects of domain classes, including attributes, associations, and structural constraints, independently of any specific business domain. It is embedded in a general-purpose object-oriented programming language (OOPL), enabling domain models to be expressed directly within implementation code while preserving declarative semantics.

Although DCSL can be interpreted as a meta-language because it specifies domain models, it satisfies the defining characteristics of a DSL: (i) it targets a well-defined domain (domain class specification); (ii) it offers a restricted and domain-focused modeling vocabulary (e.g., `DClass`, `DAttr`, `DAssoc`, `DOpt`); (iii) it defines explicit semantics grounded in OCL; and (iv) it supports automation and code generation. Accordingly, DCSL is classified as a horizontal internal DSL embedded in an OOPL.

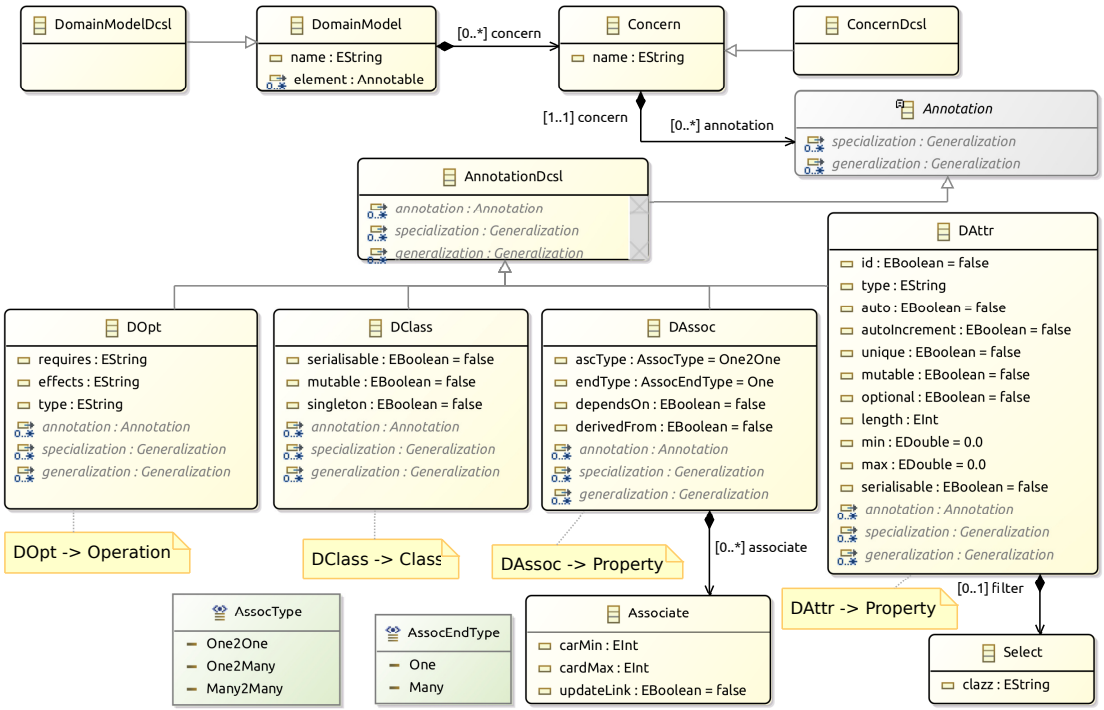

Figure 2: The DCSL metamodel (adapted from [22]).

DCSL is defined by the metamodel illustrated in Fig. 2. The metamodel is OOPL-oriented and conceptually aligned with the UML class diagram metaconcepts[8]. To support modularization and systematic composition of domain concerns, three foundational metaconcepts are introduced: `DomainModel`, `Concern`, and `Annotation` [22].

A `DomainModel` acts as the root container and aggregates `Annotable` elements and `Concerns`[1]. An `Annotable` element corresponds to an instance of a UML metaclass from class diagrams, including `Package`, `Class`, `Property`, `Association`, and `Operation`. DCSL extends these structural elements through annotations without redefining their UML semantics.

A `Concern` comprises a set of `Annotations` that encode domain-specific semantics. This design promotes explicit separation of concerns and enables the systematic composition of structural and constraint-related aspects into a unified and executable domain model.

Within the DCSL metamodel, the metaconcepts `DomainModelDcsl`, `ConcernDcsl`, and `AnnotationDcsl` specialize the generic `DomainModel`–`Concern`–`Annotation` structure. The structural constructs `DClass`, `DAttr`, `DAssoc`, and `DOpt` are concrete subclasses of `AnnotationDcsl`. Conceptually, they correspond to UML elements as follows: `DClass` to `Class`, `DAttr` to `Property`, `DAssoc` to `Property`, and `DOpt` to `Operation`. These constructs are DSL-level annotation mechanisms rather than UML stereotypes, thereby enabling executable model-to-code generation while remaining conceptually aligned with UML.

DCSL also supports the specification of essential OCL constraints [2]. Table 1 summarizes the supported structural constraints.

[1]A Concern is defined as a relatively independent aspect of a domain that encapsulates a coherent set of requirements or properties to be modeled with complete and well-defined semantics. Each concern is typically specified using a dedicated DSL, whose syntax and semantics are explicitly tailored to the particular domain aspect it addresses (e.g., structural or behavioral).

Table 1: The essential structural constraints (adapted from [20])

Constraints	Type	Descriptions
Object mutability	Boolean	Whether or not the objects of a class are mutable [26]
Field mutability	Boolean	Whether or not a field is mutable (*i.e.* its value can be changed) [27]
Field optionality	Boolean	Whether or not a field is optional (*i.e.* its value needs not be initialised when an object is created) [27]
Field uniqueness	Boolean	Whether or not the values of a field are unique [27]
Id field	Boolean	Whether or not a field is an object id field [27]
Auto field	Boolean	Whether or not the values of a field are automatically generated (by the system) [27]
Field length	Non-Boolean	The maximum length (if applicable) of a field (*i.e.* the field's values must not exceed this length) [27]
Min value of a field	Non-Boolean	The min value (if applicable) of a field (*i.e.* the field's values must not be lower than it) [27]
Max value of a field	Non-Boolean	The maximum value (if applicable) of a field (*i.e.* the field's values must not be higher than this) [27]
Min number of linked objects	Non-Boolean	The minimum number of objects to which every object of a class can be linked [8]
Max number of linked objects	Non-Boolean	The maximum number of objects to which every object of a class can be linked [8]

Example. Figure 3 illustrates a portion of the CourseMan domain model specified using DCSL. The `Student` class is annotated with `DClass`. The `name` field is annotated with `DAttr`, specifying that it is mandatory and constrained by a maximum length.

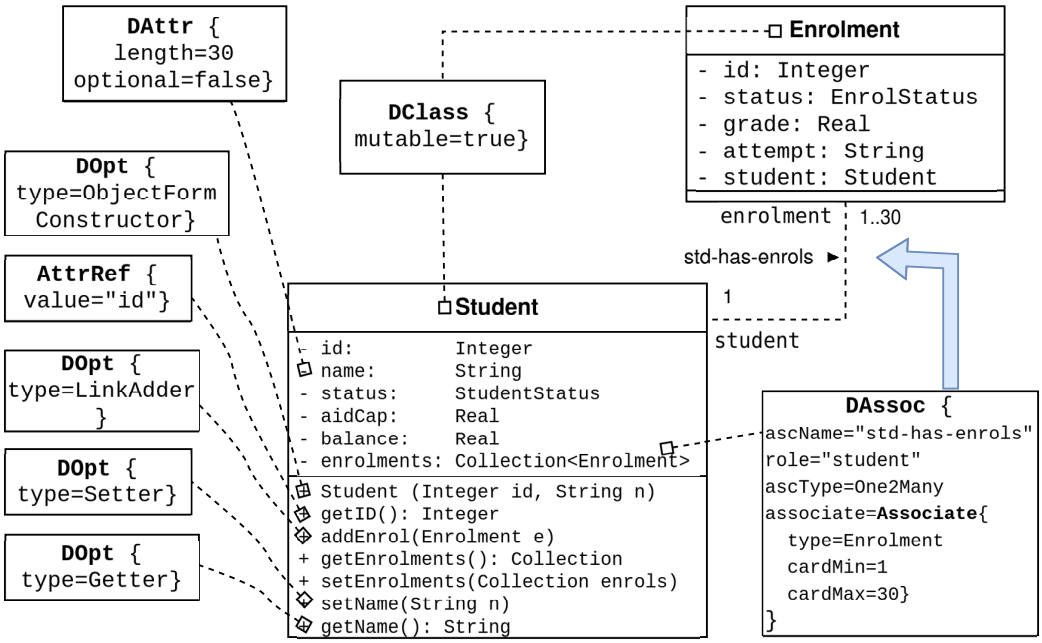

Figure 3: A simplified representation of the **CourseMan** domain model using DCSL.

Figure 3 illustrates a portion of the CourseMan domain model specified using the DCSL. This model includes two domain classes: `Student` and `Enrolment`, both annotated with the `DClass` element, indicating that they are mutable domain classes (`DClass.mutable=true`). Specifically, the `Student` class contains three domain fields: `id`, `name`, and `enrolments`. The `name` field is annotated with a `DAttr` element, specifying that it is mandatory (`DAttr.optional=false`) and has a maximum length of 30 characters (`DAttr.length`=30), among other attributes.

In previous work [20], DCSL focused on representing essential structural constraints at the class and attribute level, with a predefined and fixed constraint vocabulary.

In the present work, DCSL is extended with Constraint Annotation Patterns (CAPs), which introduce a reusable and parameterized abstraction layer for complex OCL invariants. This extension transforms DCSL from a structural annotation language into a pattern-augmented modeling framework supporting systematic constraint composition within a unified domain model.

3. Overview of Our Approach

Figure 4 provides an overview of our approach, which consists of two main phases: *Phase 1* focuses on defining a catalog of constraint annotation-based patterns (CAPs) by eliciting and generalizing them from available domain requirements (Step 1 in Fig. 4). *Phase 2*, comprising the remaining three steps in Fig. 4, aims to derive a final software prototype that satisfies the input requirements of the target system. The main steps of our approach are summarized as follows.

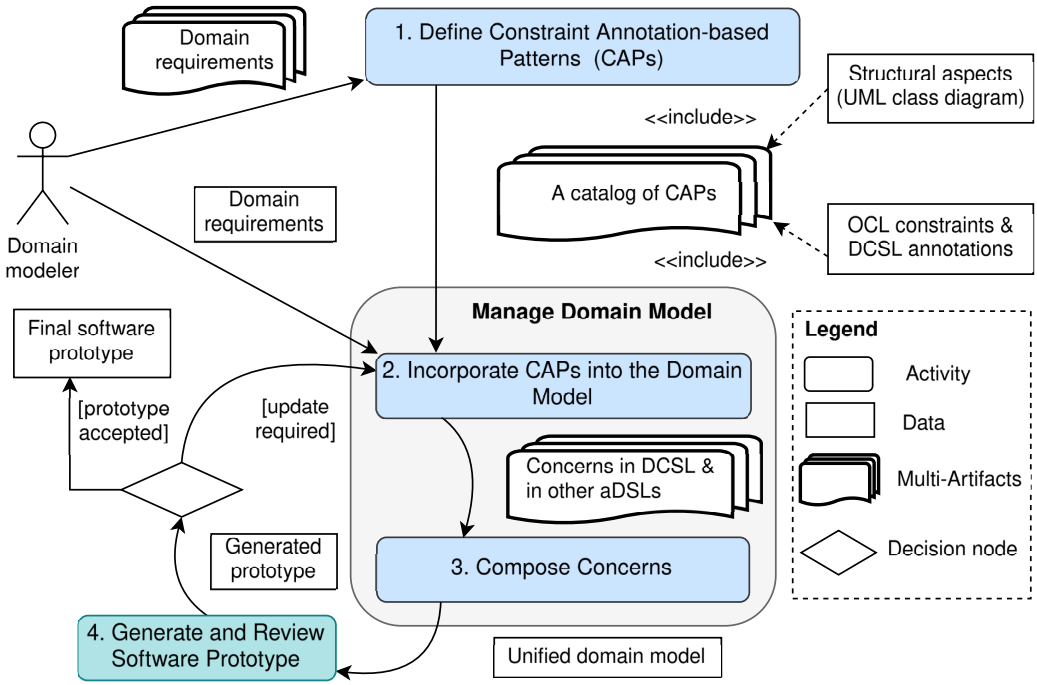

Figure 4: Overview of our approach.

Step 1 – Define CAPs: Based on a collection of domain requirements from different domains, this step aims to identify groups of OCL constraints, generalize each group, and derive a common pattern, called a CAP, to represent the OCL constraints belonging to that group. Each CAP encapsulates: (i) a UML class diagram to capture domain concepts and their relationships, (ii) an OCL specification of business rules and invariants, and (iii) annotations representing the OCL constraints.

Step 2 – Incorporate CAPs into the domain model: The input of this step consists of a set of OCL domain constraints and the existing catalog of CAPs. This step aims to represent the OCL constraints in terms of CAPs in order to constrain the domain model. Each CAP-based constraint is expressed through an extension of DCSL.

Step 3 – Compose concerns for a unified domain model: All relevant concerns covering both structural and behavioral aspects of the domain model, represented by aDSLs such as DCSL, are subsequently composed into a unified domain model.

Step 4 – Generate and review the software prototype: The unified domain model serves as the blueprint for the automatic generation of prototypes. The JDA framework can be employed to perform this generation. The resulting prototype is then delivered to stakeholders for evaluation. If feedback is provided, the managed domain model is updated accordingly. The process then repeats, supporting a continuous improvement cycle until the software prototype satisfies the specified requirements.

4. Incorporation of CAPs into Domain Models

This section presents our method for specifying and managing CAPs. We extend DCSL to support the representation of CAP-based constraints. Furthermore, we define an initial catalog of CAPs, which enables the expression and integration of OCL constraints into domain models, thus producing an executable and unified domain model.

4.1. Syntax Aspect

The basic idea of a CAP is to use a template to represent OCL constraints within a domain model. Essentially, a CAP is a parameterized template. Each application of a CAP to represent a specific OCL constraint corresponds to assigning concrete values to the pattern's parameters. The input parameters of each template are defined through an annotation mechanism. In practice, an OCL constraint can be parameterized in several ways. The most general approach is to follow the keywords defined in the OCL syntax. However, this approach may be difficult for domain experts who are not familiar with OCL. Therefore, in our approach, we adopt a parameterization strategy based on the semantics of the constraint, where parameters are identified according to the meaning of the constraint within the considered domain model. This design choice enables the annotations to better capture the semantic intent of the constraints in the target domain model. We extend DCSL to represent CAPs, allowing each CAP to be formally specified and managed within the same modeling framework. From a syntactic perspective, each CAP pattern is specified with the following main components:

Pattern Name: Each CAP is assigned a unique identifier to facilitate its management and retrieval from the existing CAP catalog.

Description: Provides a brief explanation of the purpose and semantics of the OCL constraints represented by the CAP.

Template: Describes the parameterized OCL expression template, which conforms to the OCL syntax and corresponds to a specific group of OCL constraints sharing the same structural form. In particular, it includes the following specification elements:

- *Class diagram structure* — describes the relevant portion of the class diagram that defines the context of the OCL template. This structure includes parameters that specify class names, attributes, and relationships.
- *OCL template* — represents the parameterized OCL expression of the pattern.
- *Annotation specification* — define the parameters through a structured annotation mechanism used in the OCL expression of the pattern.

Example: Provides a concrete illustration of how the CAP is applied.

In summary, CAPs provide a reusable, parameterized mechanism for representing recurring OCL constraints within domain models. Accordingly, Definition 1 summarizes the notion of CAP.

Definition 1 (Constraint Annotation Pattern)**.** A *Constraint Annotation Pattern* (*CAP*) is defined as a parameterized template that captures the common syntactic and semantic structure of a class of OCL constraints. Formally, a CAP can be defined as a tuple:

$$CAP = (N, D, \mathcal{T})$$

where:

- N is the name of the pattern.
- D is a textual description explaining the purpose and semantics of the OCL constraints represented by the pattern.
- $\mathcal{T}$ is a set of tuples $\langle P, T \rangle$, where P is a set of parameters representing the variable elements of the template (e.g., attributes, associations, or values) specified through annotations, and T is a parameterized OCL expression template describing the structural form of the OCL constraint.

An instance of a CAP is obtained by substituting the parameters P with concrete elements of the domain model, resulting in a valid OCL constraint within the given context.

4.2. An Example and the First CAP Catalog

In this section, we introduce a CAP pattern named SumConstraint to illustrate our proposed approach. We then describe the first catalog of CAPs that we have collected and specified from various domain sources.

Pattern Name: SumConstraint

Description: This pattern defines OCL constraints for controlling and managing quantities, values, or resources through aggregation operations. Specifically, this pattern ensures that the aggregate value stored in an entity always equals the total of its related components. It is used to prevent discrepancies between derived data and source data, thereby supporting reconciliation and auditing.

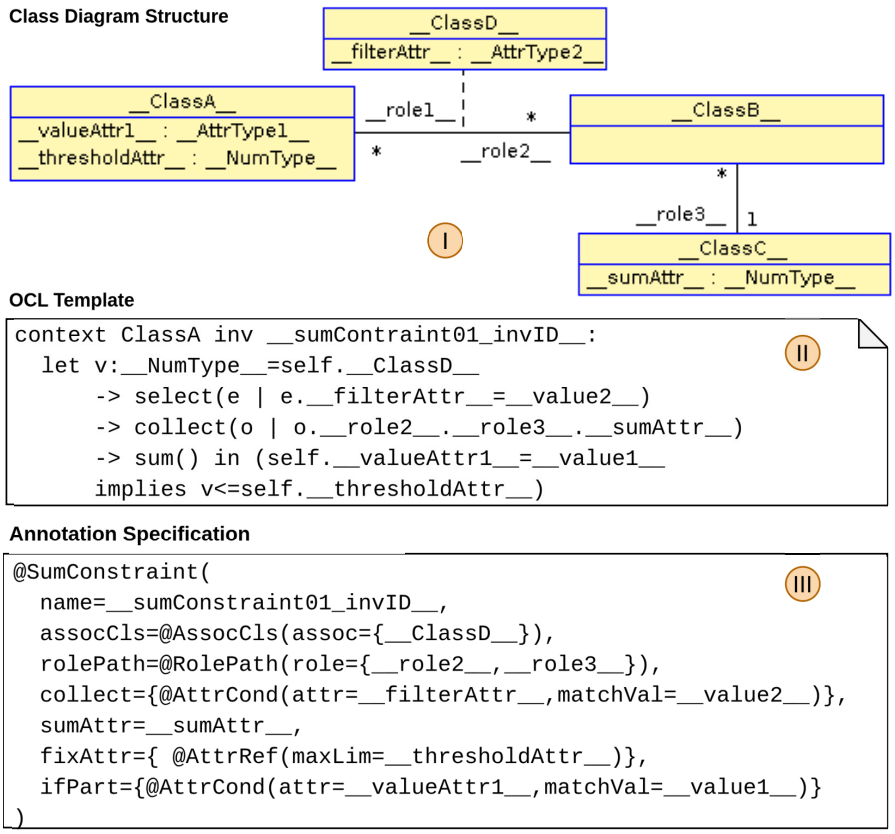

Figure 5: Template specification of the CAP pattern `SumConstraint`.

Template: The parameterized OCL expression of this pattern is defined based on the following main source.

- *Class diagram structure:* Figure 5 (label I) shows a class diagram depicting three classes, their attributes, and the associations between them. This diagram defines the context of the underlying OCL constraint template.
- *OCL template:* Figure 5 (label II) illustrates the OCL template used to generate constraints that express the following restriction: The sum of the values of the attribute `__sumAttr__` over all objects `o` in a collection filtered from the set of `__ClassD__` objects associated with the current `__ClassA__` instance (`self`) is computed according to the filtering criterion defined by the variable `__filterAttr__`. The computed sum is then validated against `__thresholdAttr__`, to ensure that the threshold condition is satisfied and that objects do not exceed the permitted limit. Note that the names of the variables in the template begin and end with "__". The variable name `__sumConstraint01_invID__` implies that this is OCL template type 01 of the SumConstraint pattern.
- *Annotation specification:* We extend DCSL with new annotations, as illustrated in Fig. 5 (label III), to capture the parameters of the OCL template. The annotation specification enables the generation of an OCL constraint once the parameters are assigned concrete values.

Example: We focus on the **CourseMan** model, as shown in Fig. 1, to illustrate this pattern. The underlying constraint ensures that the maximum number of credits a student can register for is limited to 12 credits per semester when the student is under

academic warning. This helps the student focus on improving their GPA. The constraint is expressed in OCL as follows:

```
context Student
inv courseMan_inv06_ProbationRules:
  let v:Integer=self.enrolment
    ->select(e|e.status=EnrolStatus::ACTIVE)
    ->collect(e|e.offering.module.credits)->sum()
  in self.overallStatus=AcademicStatus::PROBATION
    implies v<=12
```

We define the specification in DSCL to generate the constraint as follows:

```
@SumConstraint(
  name='courseMan_inv06_ProbationRules',
  assocCls=@AssocCls(assoc={'Enrolment'}),
  rolePath=@RolePath(role={'offering','module'}),
  collect={@AttrCond(attr='status',
    matchVal='EnrolStatus::ACTIVE')},
  sumAttr='credits',
  fixAttr={@AttrRef(maxLim = 12)},
  ifPart={@AttrCond(attr='overallStatus',
    matchVal='AcademicStatus::PROBATION')}
)
class Student {...}
```

A CAP pattern represents a semantic family of structurally related OCL invariants. Each CAP may contain multiple parameterized OCL template variants that correspond to different syntactic forms of constraints within the same semantic category.

For example, the SumConstraint pattern contains several OCL template variants corresponding to different groups of aggregation-based constraints. The OCL template illustrated in Fig. 5 represents one such variant. Other variants may capture similar constraints with additional filtering conditions or contextual predicates.

Although these template variants share the same pattern name, they differ in their structural formulation. During modeling, the appropriate template variant is selected, and its parameters are instantiated via a DCSL annotation.

Users formulate constraints by instantiating the corresponding CAP annotation with appropriate parameter values. The concrete OCL invariant is then automatically generated from the selected template variant.

The SumConstraint pattern contains several OCL template variants corresponding to different groups of OCL constraints. The OCL template illustrated in Fig. 5 is one such variant. For example, consider another OCL constraint on the domain model shown in Fig. 1, as described below. This constraint ensures that students with a GPA below 2.5 in the previous semester are allowed to enroll in a maximum of 20 credits.

```
context TermRecord
inv courseMan_inv17_LowGPACreditRestriction:
    self.prevSemGpa<2.5 implies
```

```
      self.student.enrolment->select(e| e.offering.term=self.term)
      ->collect(e|e.offering.module.credits)->sum()<=20
```

The OCL template of the SumConstraint pattern to generate this constraint is defined as follows:

```
@SumConstraint(
  name='courseMan_inv17_LowGPACreditRestriction',
  assocCls=@AssocCls(assoc={'Enrollments'}),
  rolePath=@RolePath(role={'student,'offering','module'}),
  rolePath2=@RolePath(role={'term','termRecord'}),
  collect={@AttrCond(attr='curTerm',matchVal='true')},
  sumAttr='credits',
  fixAttr={@AttrRef(maxLim=20)},
  ifPart={@AttrCond(attr='prevSemGpa',maxLim=2.5)}
)
class TermRecord { ... }
```

In this example, `rolePath` reaches the aggregated `credits`, while `rolePath2` provides the term reference that enables selecting only enrolments whose `offering.term` corresponds to the record's term. The `rolePath2` parameter denotes a secondary navigation chain required by certain `SumConstraint` template variants to bind an auxiliary context object (e.g., the current term of `TermRecord`) used for matching or filtering.

Although this constraint differs syntactically from the template shown in Fig. 1, it belongs to the same semantic family of conditional aggregation constraints. Therefore, it can be represented as another template variant within the SumConstraint pattern.

We identified a CAP catalog that includes ten patterns, each formally specified and managed within a unified modeling framework. Each CAP represents a family that captures the shared semantic intent of constraints of the same kind, generalizing multiple domain rules with similar structures. Each CAP includes a set of types that distinguish between different OCL template variants of the CAP pattern. The two example OCL invariants discussed in this section are generated by the two OCL templates of the SumConstraint pattern. A comprehensive description of the CAP patterns included in our catalog is provided in the accompanying technical report [28]. The report is available as an open-source project on GitHub[2].

4.3. Semantics Aspect

CAP templates are defined as parameterized OCL invariants, whereas DCSL annotations provide the parameter values for their instantiation in a domain model. To represent OCL constraints within CAP, we distinguish between two complementary mapping mechanisms. Direct annotation mapping applies to essential structural constraints that can be expressed directly through DCSL annotations without introducing a template abstraction. In contrast, template-based instantiation is used for more complex OCL invariants that follow recurring patterns. In this case, the invariant is captured as a parameterized CAP template, while DCSL annotations provide the parameter assignments required for its instantiation.

[2]https://github.com/vinhskv/CAP_UDML/tree/main/longVersion

In the mapping from OCL constraints to CAP annotations, the structural elements of an OCL invariant are encoded as annotation parameters. Navigation expressions are represented through role-path parameters specifying association paths in the domain model. Filtering conditions are encoded as conditional parameters, aggregation operators correspond to the aggregation attributes, and comparison thresholds are represented as bound values associated with constraint attributes. This parameterization preserves the structural semantics of the original OCL constraint while enabling its executable realization in the annotated domain model.

The semantics of CAP correspond to those of OCL through a one-to-one mapping between each CAP instance and the generated OCL invariant in the domain model.

Definition 2 (**CAP Application**)**.** Let CD be a class diagram representing the domain model; Let OCL_{CD} denote the set of all valid OCL invariants expressible over CD; Let CAP_i be a CAP defined by a parameterized OCL template T_i with parameter space $\mathcal{D}_i$. A *CAP application* on CD is the process of binding a parameter assignment $d_i \in \mathcal{D}_i$ to T_i so that the OCL template of the CAP becomes a valid OCL invariant on CD. Formally, this process is represented by the *generation mapping*

$$\mathcal{G} : (CAP_i, d_i) \longmapsto oclInv \in OCL_{CD},$$

where $\mathcal{G}(CAP_i, d_i)$ denotes the specific OCL invariant generated by instantiating the CAP template T_i with parameters d_i.

Example. Consider the constraint `courseMan_inv06_ProbationRules`. This constraint is generated by applying the CAP SumConstraint, as explained in Subsection 4.2. The corresponding generation mapping is defined as follows.

- CD represents the domain model CourseMan, as depicted in Fig. 1.
- CAP_i is the pattern SumConstraint in which:
  - T_i is the template of the CAP as shown in Fig. 5.
  - $\mathcal{D}_i$: The parameter space defined on CD; $d_i \in \mathcal{D}_i$ denotes the parameter assignment that corresponds to the annotation specification for the invariant `courseMan_inv06_ProbationRules`, as detailed in Subsection 4.2.

The OCL invariant is structurally decomposed into the following CAP parameter assignments:

- Navigation path → `rolePath` = {'offering', 'module'}
- Filter condition → `collect` = {`@AttrCond`(`attr` = 'status', `matchVal` = 'ACTIVE')}
- Aggregation attribute → `sumAttr` = 'credits'
- Threshold value → `fixAttr` = {`@AttrRef`(`maxLim` = 12)}

The resulting annotation constitutes an instantiation of the SumConstraint template. By applying the generation function $\mathcal{G}(CAP_i, d_i)$, the corresponding concrete OCL invariant is reconstructed.

Definition 3 (**CAP Catalog Completeness**)**.** Let $\mathcal{C}_{CAP}$ be a set of CAPs. The $\mathcal{C}_{CAP}$ is said to be *complete* with respect to the CD domain model if, for every invariant $oclInv \in OCL_{CD}$, there exists a $CAP_i \in \mathcal{C}_{CAP}$ and a parameter assignment $d_i \in \mathcal{D}_i$ such that $\mathcal{G}(CAP_i, d_i) = oclInv$.

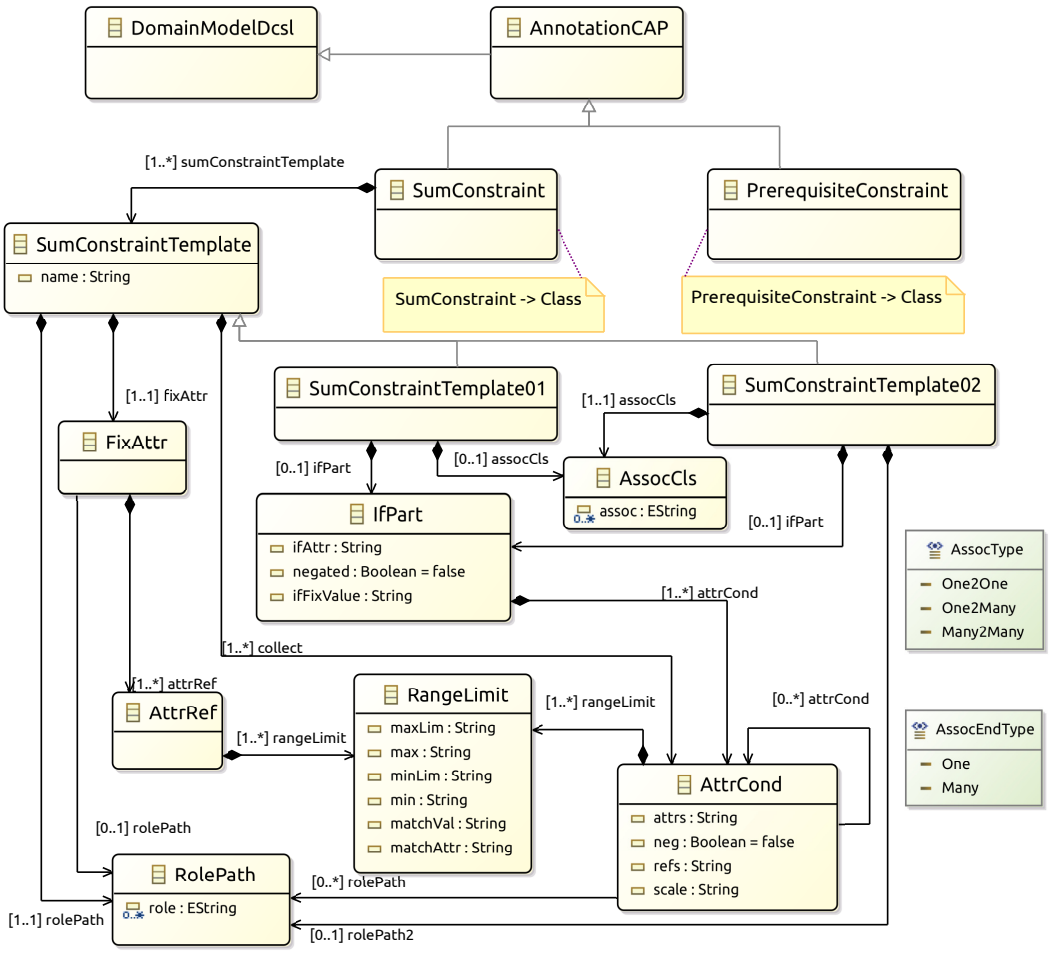

Figure 6: Extension of the DCSL metamodel to represent CAPs.

Figure 6 presents the CAP abstract syntax at the metamodel level, illustrating how DCSL is structurally extended to support CAPs and defining CAP types rather than concrete constraint instances. The core extension point is the metaconcept `AnnotationCAP`, which specializes `AnnotationDcsl`. Concrete CAP types, such as `PrerequisiteConstraint` and `SumConstraint`, are defined as metaclasses inheriting from `AnnotationCAP`. These metaclasses represent CAP annotation types at the metamodel level, not domain-level constraint instances.

Each CAP metaclass is structurally associated with a set of template variant types (e.g., `SumConstraintTemplate01`, `SumConstraintTemplate02`) that capture structural variants of the pattern. These templates define parameter structures corresponding one-to-one to the attributes of the related CAP annotation. Auxiliary metaconcepts (e.g., `AssocCls`, `RolePath`, `AttrCond`, `RangeLimit`) specify how the corresponding OCL invariants are systematically constructed.

Conceptually, CAP is organized into three abstraction levels: (i) the metamodel level, which defines CAP annotation types and their template structures (Fig. 6); (ii) the pattern catalog level, which provides parameterized OCL templates associated with each CAP type; and (iii) the modeling level, where concrete CAP annotations are instantiated in domain models and transformed into OCL invariants through the generation mapping.

A CAP model at the modeling level therefore conforms to the CAP metamodel defined in Fig. 6. The figure thus specifies the extension structure and abstract syntax of CAP, while concrete constraint instances appear only at the modeling level.

Importantly, the metamodel does not need to be modified when CAP annotations are instantiated or reused in domain models. Metamodel extension is required only when introducing a new CAP family with a distinct annotation structure. In this sense, the CAP catalog is conceptually open and extensible, while the metamodel provides the structural definition for CAP types rather than for individual constraint instances.

4.4. Applying CAPs and Generating Software Prototypes

This subsection describes the methodological workflow for applying Constraint Annotation Patterns (CAPs) within a domain-driven modeling process and for deriving executable software prototypes from a Unified Domain Model (UDM). The workflow focuses on the conceptual transformation from CAP-based constraint specifications into executable domain models. Figure 7 illustrates the overall workflow.

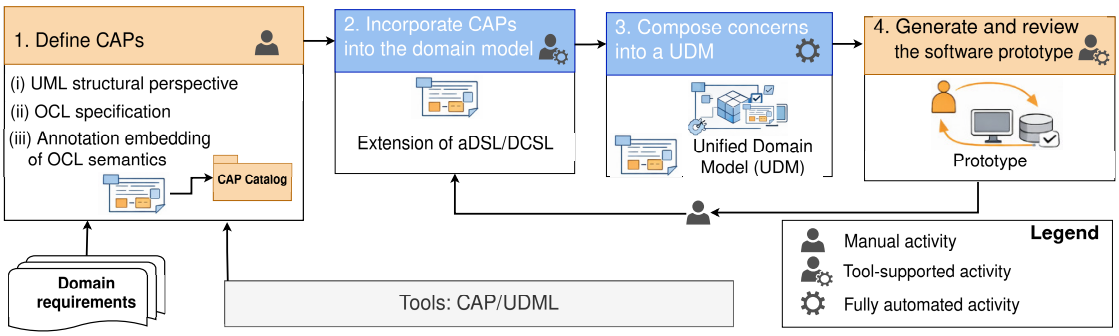

Figure 7: CAP-based workflow from constraint pattern definition to prototype generation, combining manual, semi-automated, and automated stages.

The workflow consists of four logically connected stages that progressively transform domain constraints into executable artifacts. Depending on the stage, activities may involve manual modeling tasks, semi-automated configuration steps, or fully automated transformations as summarized in Figure 7.

Step 1 – CAP definition and catalog construction. The first stage focuses on identifying reusable constraint structures and organizing them into a CAP catalog. This activity is primarily a manual modeling task performed by domain modelers or language designers. CAPs are derived by analyzing recurring OCL invariants observed across multiple domain models and generalizing them into reusable constraint families.

Each CAP entry formally captures a constraint pattern through three complementary elements:

- a UML structural perspective describing the relevant modeling context,
- a parameterized OCL template specifying the semantic structure of the invariant, and
- an annotation definition that specifies the parameter space for pattern instantiation.

Once defined, CAP templates can be validated against their structural context to ensure semantic consistency. The resulting CAP catalog forms a reusable repository of constraint templates that can be applied across different domain models.

Step 2 – CAP instantiation within the domain model. In the second stage, CAPs are instantiated within a domain model expressed using an annotation-based domain-specific language. This stage is a semi-automated activity that combines manual specification by the modeler with automated interpretation by the modeling environment.

Instead of writing explicit OCL invariants directly, the modeler specifies domain constraints declaratively using CAP annotations (e.g., `@SumConstraint`. Each annotation corresponds to a concrete parameter assignment $d_i \in \mathcal{D}_i$ for a CAP template T_i, as defined by the formal CAP application mapping introduced in Definition 2.

Structural aspects of the domain are represented using UML class definitions, while constraint semantics are specified through CAP annotations that bind domain elements—such as attributes, associations, and threshold values—to the parameters of the corresponding CAP template. This approach allows domain constraints to be expressed declaratively at the model level without requiring direct manipulation of OCL syntax.

Step 3 – OCL reconstruction and composition into a UDM. After CAP annotations are specified, the corresponding OCL invariants are reconstructed automatically through the generation mapping defined in Section 4.3. This stage is fully automated.

For each CAP instantiation (CAP_i, d_i), the parameterized OCL template T_i is instantiated to produce a concrete invariant: $G(CAP_i, d_i) \longmapsto oclInv$. The generated invariants are integrated into the domain model together with the structural definitions, producing a Unified Domain Model (UDM). The UDM therefore combines two complementary elements:

- structural specifications of domain entities and relationships, and
- formal OCL invariants reconstructed from CAP templates.

This automated reconstruction step preserves the formal semantics of OCL while enabling constraints to be specified through higher-level pattern abstractions. The resulting model can then be validated using standard OCL validation mechanisms to ensure syntactic correctness and consistency with the domain structure.

Step 4 – Prototype generation and review. The validated UDM serves as the input for model-driven prototype generation. From this model, a generation framework can automatically derive executable software artifacts, including domain classes, constraint validation logic, persistence configurations, and user interface components.

The generated prototype provides an operational representation of the domain model and enables early validation of domain rules and system behavior. Prototype evaluation typically proceeds at two complementary levels. First, structural and constraint-level correctness are verified through automated model validation. Second, domain experts and stakeholders review the generated prototype to confirm that the implemented behavior conforms to the intended business rules.

Feedback obtained during this evaluation may lead to refinements of the domain model or adjustments to CAP instantiations. This iterative process supports continuous alignment between domain knowledge, constraint specifications, and executable software artifacts.

5. Tool Support

This section describes the tool support developed to operationalize the proposed CAP-based modeling approach. The toolchain enables CAP definition and management, annotation-based constraint specification, automatic reconstruction of OCL invariants, and generation of executable software artifacts from a Unified Domain Model (UDM). Together, these capabilities ensure a seamless transition from formal constraint patterns to runnable domain-driven prototypes.

5.1. Tool Architecture: The CAP/UDML Extension of USE

In this subsection, we present the methodology for developing the supporting tool, called CAP/UDML. The tool is implemented as an extension of the UML-based Specification Environment (USE) [30], augmenting its native support for UML structural modeling and OCL constraint validation with annotation-based CAP specification and automatic invariant reconstruction.

To enable the composition of CAPs into a UDM, CAP/UDML extends the USE grammar and processing pipeline to (i) recognize CAP annotations, (ii) associate them with cataloged CAP templates, and (iii) automatically generate corresponding OCL invariants according to the formal generation mapping defined in Section 4.3. The generated invariants are then validated using the underlying USE OCL engine to ensure syntactic and structural consistency with the domain model.

The implementation is available as an open-source project on GitHub[3].

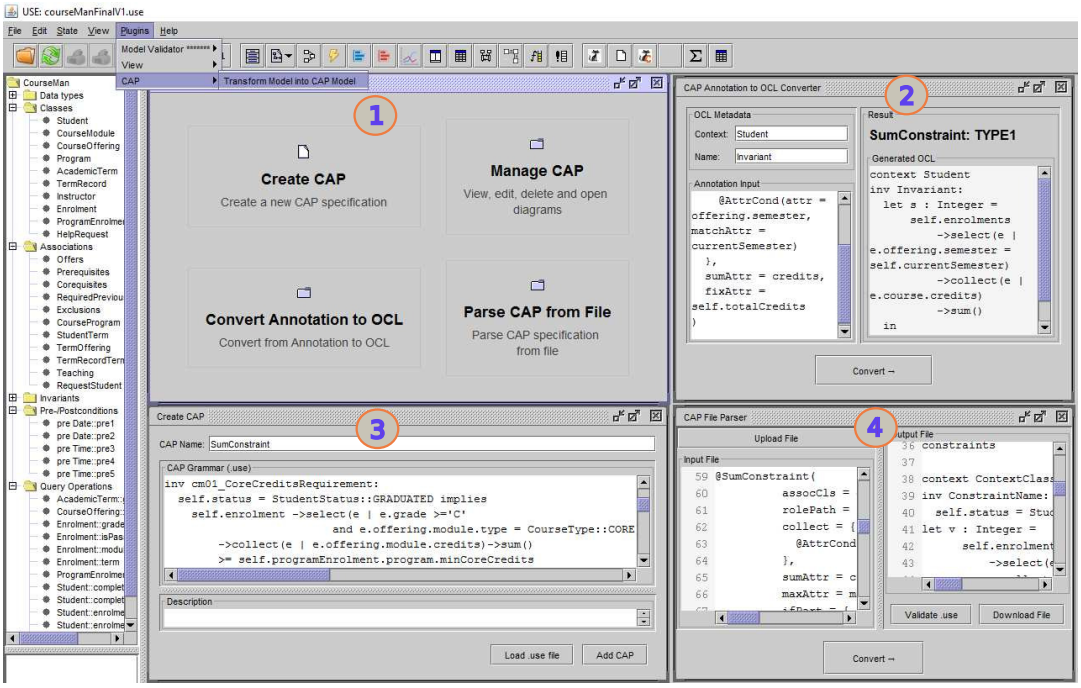

Figure 8: CAP/UDML supporting tool for CAP management and OCL reconstruction

[3]https://github.com/vinhskv/CAP_UDML/tree/main/CAP_UDML

Specifically, Fig. 8 provides an architectural overview of the CAP/UDML tool, illustrating the extension layers built on top of USE and the workflow from annotation parsing to OCL reconstruction and UDM composition.

At the top-left (label (1)) of Fig. 8, the interface shows the four main functions of the CAP/UDML tool, including: (i) Create CAP, (ii) Convert Standalone Annotation to CAP, (iii) Parse Annotations from File (`.use context`), and (iv) Manage CAP: Each CAP may contain multiple types. This function allows users to define and manage these types within a predefined CAP.

At the top-right (label (2)) of Fig. 8, the interface presents the **Convert Standalone Annotation to OCL** function. This function focuses on parsing standalone annotations, i.e., annotations that are not associated with a specific model context. To transform such annotations into OCL constraints, the user must manually define the context (e.g., the class in which the annotation should be applied, such as `Student` or `CourseModule`) and provide an invariant name, which will serve as the identifier of the generated OCL constraint.

At the bottom-left (label (3)) of Fig. 8, the interface shows the **Create CAP** function. This function allows users to define a new CAP type. When creating a CAP, the user specifies the CAP name (e.g., SumConstraint) and provides a corresponding `".use"` file that illustrates the class diagram structure associated with that CAP. The system allows the `".use"` file to be validated to ensure that its grammar and syntax conform to the USE specification.

At the bottom-right (label (4)) of Fig. 8, the interface shows the **Parse Annotations from a `".use"` File** function. This function processes annotations that already appear within a `".use"` model file. When such a file is loaded, the plugin analyzes the annotations together with their model context, reconstructs the corresponding OCL constraints, and generates a new `".use"` file in which the annotations are automatically converted into these constraints. The reconstructed constraints are then integrated into the UDM. After generation, the resulting `".use"` file can be validated to ensure that its grammar and syntax are correct.

5.2. Software Artifact Generation

Once the UDM has been validated, it is provided as input to the prototype generation layer implemented using Spring Boot and the JDA framework [29]. The generation framework is available as an open-source project on GitHub[4].

Based on the UDM, JDA automatically generates executable artifacts, including domain classes, validation logic derived from reconstructed OCL invariants, persistence configurations, and graphical user interface components.

As illustrated in Fig. 9, the structural model comprises module classes and their corresponding domain classes, annotated with CAP-based constraint specifications such as `@SumConstraint` and `@PrerequisiteConstraint`. These annotations have been previously transformed into concrete OCL invariants during the UDM composition stage, and the generated prototype enforces the corresponding validation logic at runtime.

[4]https://github.com/vinhskv/CAP_UDML/tree/main/frameWorkGenCode

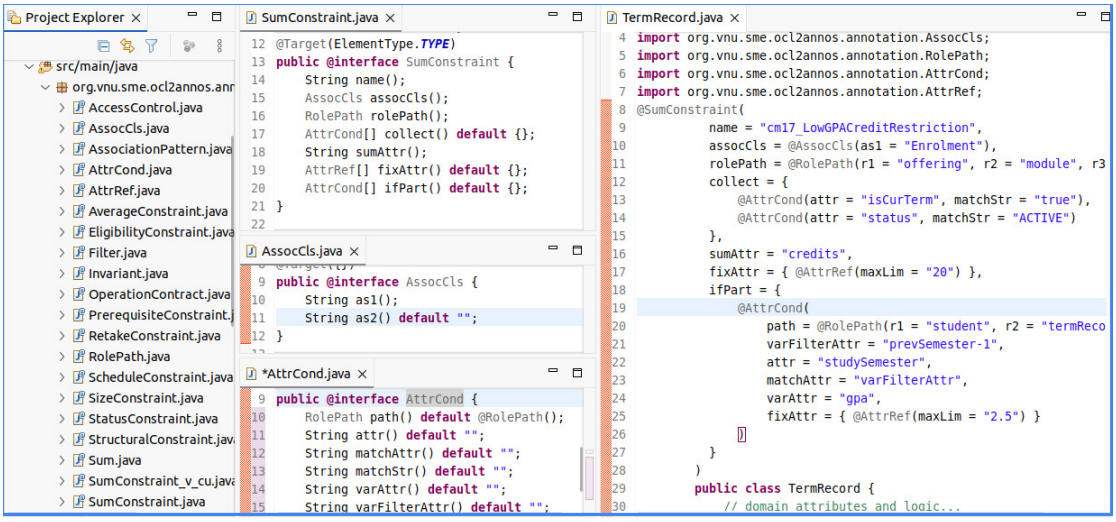

Figure 9: Tool support realization and usability of a CAP framework.

The generated CourseMan application with a graphical user interface (GUI) is presented in Fig. 10. This example demonstrates the runtime validation mechanism for enforcing the credit-limit constraint: the total number of registered credits for a student must not exceed 15. At the top of the figure, (1) shows a valid case where the student "Tran Van Anh" has registered for 14 credits, whereas (2) illustrates the validation error raised when attempting to exceed the credit limit. When performing the save operation to enroll in a new course, the system automatically activates the validation mechanism according to the method we proposed.

6. Evaluation and Discussion

This section reports a systematic evaluation of CAP and discusses key findings. Following the DSL evaluation dimensions in [31], we assess CAP along three dimensions—*expressiveness*, *required coding effort*, and *effectiveness*—and further examine its applicability on heterogeneous domain systems.

6.1. Research Questions

Our evaluation addresses the following research questions:

RQ1. How expressive are CAPs for modeling domain concepts compared to existing DDD methods?
RQ2. To what extent can CAP be applied in real-world software engineering practices?

6.2. Evaluation Methodology

To substantiate the evaluation results and minimize subjective interpretation, we conducted controlled reconstruction experiments comparing CAP with four platforms: DCSL [20], Apache Causeway [18], OpenXava [19], and Actifsource [32].

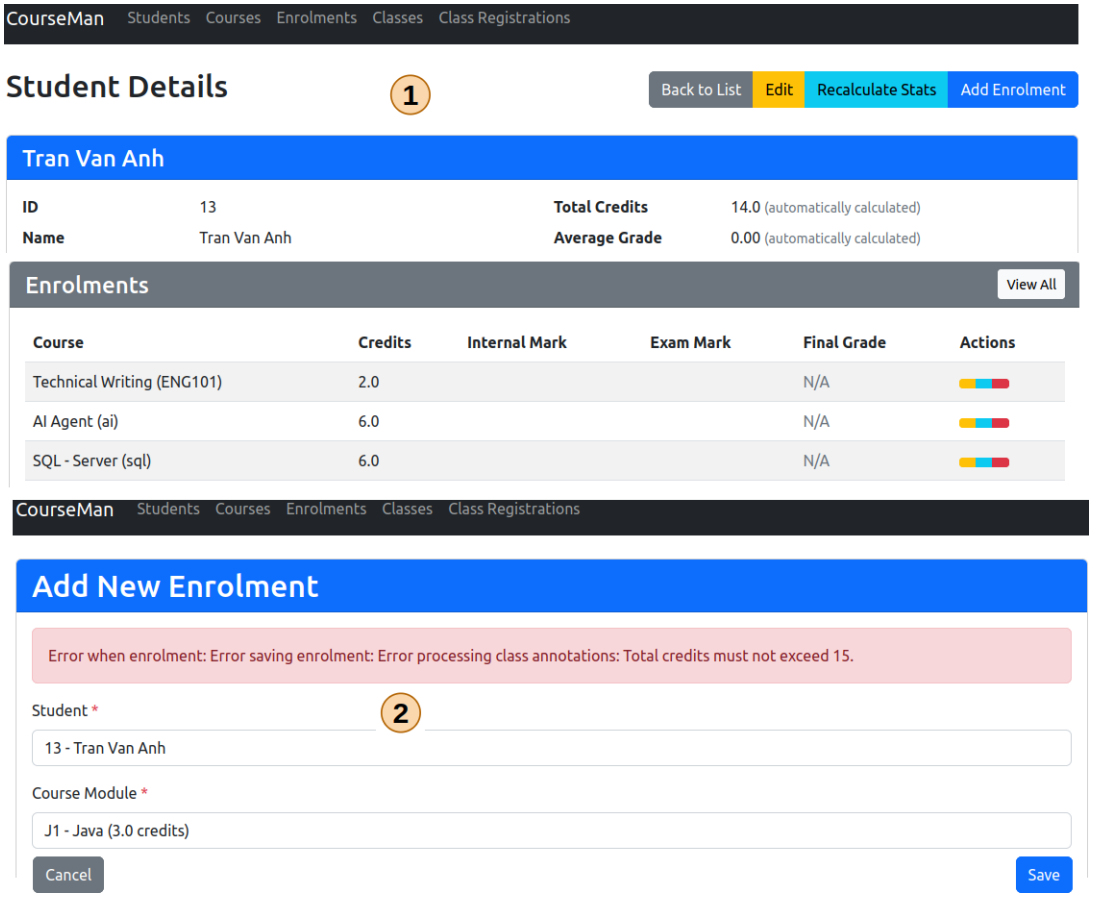

Figure 10: The GUI of the CourseMan software generated by the tool.

Case studies. For each platform, we reconstructed functionally equivalent domain models for three case studies—CourseMan, ProcessMan, and OrderMan—selected to ensure domain diversity, constraint diversity, and varying modeling maturity. CourseMan is a benchmark academic domain with rich structural and behavioral constraints. ProcessMan enables controlled comparison with prior DCSL-based modeling to assess incremental benefits of CAP. OrderMan, adapted from enterprise business-process scenarios, represents a heterogeneous, workflow-intensive domain to evaluate scalability and general applicability.

Table 2: Summary of case studies: CourseMan, ProcessMan, and OrderMan

Case Study	Domain Type	OCL Constraints	Semantic OCL Constraint Types	Role in Evaluation
CourseMan	Academic	184	14	Expressiveness benchmark
ProcessMan	Process management	140	12	Controlled DCSL comparison
OrderMan	Business workflow	76	11	Scalability & generality

Table 2 summarizes the quantitative characteristics of the three systems, including the number of constraints and the semantic OCL Constraint Types used in the evaluation.

Controlled reconstruction protocol. For each case study and each platform, we performed the following steps:

1. Reconstruct the domain model (class diagram) using the platform's native mechanisms;
2. Encode the constraints to match the reference specification as closely as possible;
3. Record measurable indicators:
   - number of constraints supported directly by annotations or built-in constructs,
   - number requiring manual implementation,
   - manual validation code size (LOC) for each constraint,
   - implementation time,
   - availability of automatic generation support (model-to-code and validation generation).
4. Validate the reconstruction results independently by two researchers; discrepancies were resolved by re-checking the encoding artifacts and reaching consensus.

All evaluation dimensions reported in the following subsections are grounded in these recorded artifacts and measurements. Detailed criterion lists, measurement tables, and case-level mapping evidence are provided in Appendix A–Appendix C.

6.3. RQ1: Expressiveness of CAP in Domain Modeling

Expressiveness evaluates the ability of each platform to specify and enforce domain constraints declaratively at the model level. We operationalize expressiveness using twelve criteria derived from recurring constraint families across the three case studies. For transparency and self-containment, we enumerate these criteria and provide criterion-level evidence in Appendix A.

Table 3: Comparison of platforms by expressiveness levels (12 criteria)

Platform	Fully supported	Partially supported	Not supported
CAP	10/12	2/12	0/12
DCSL	4/12	5/12	3/12
OpenXava	2/12	7/12	3/12
Apache Causeway	2/12	7/12	3/12
Actifsource	9/12	3/12	0/12

Results. Table 3 summarizes how many of the twelve criteria fall into each level for each platform. Appendix A reports (i) the twelve criteria, and (ii) the concrete constructs used per platform.

Under the 12 expressiveness criteria, CAP supports all categories (12/12), comprising 10 fully supported and 2 partially supported criteria, with no unsupported cases. DCSL, OpenXava, and Apache Causeway each leave three criteria unsupported (3/12). Actifsource also covers all criteria (12/12), though several criteria are supported via meta-model configuration and model-level rules rather than direct annotation-based constructs.

6.4. Required Coding Effort

This evaluation focuses exclusively on the *essential structural OCL constraints* defined in Table 1 (Subsection 2.3), which summarizes the set of core domain constraints considered

in the study. The objective is to quantify the handwritten implementation effort required to enforce these constraints when using the CAP-extended DCSL compared to other platforms.

Evaluation Scope. For each of the three case studies (CourseMan, ProcessMan, and OrderMan), we reconstructed identical domain models and implemented the same set of essential OCL constraint categories listed in Table 1 (Subsection 2.3), including:

- attribute value constraints,
- multiplicity and cardinality constraints,
- dependency constraints,
- aggregation-based constraints (e.g., sum/count),
- cross-association structural invariants.

Only essential constraints were included in the measurement, as they represent the core domain integrity rules targeted by CAP. Auxiliary and framework-level constraints were excluded because they are typically handled automatically and would therefore not provide a meaningful basis for evaluating the reduction in manual modeling effort.

Measurement Protocol. For each platform and each essential constraint instance, we recorded:

1. whether the constraint could be enforced declaratively without handwritten validation code;
2. the number of handwritten validation methods required;
3. the number of handwritten validation LOC (excluding comments and blank lines);
4. the number of auxiliary validator/service classes introduced solely for enforcement.

Note that, structural class declarations were not counted as validation code because they belong to the domain model specification rather than to constraint validation logic, which is the focus of CAP.

Table 4: Required coding effort for essential OCL constraints

Platform	**Avg manual LOC/-constraint**	**% auto-enforced essential constraints**	**Additional validator classes**
CAP-extended DCSL	6.2	92%	0
Apache Causeway	54.3	22%	6
OpenXava	37.5	45%	4
Actifsource	31.8	52%	3

Quantitative Results. Table 4 summarizes the aggregated results averaged over the three case studies. The complete measurement protocol and supporting evidence are presented in Appendix B.

Analysis. The CAP-extended DCSL achieves the highest automation rate (92%) for essential OCL constraints and requires the lowest average handwritten validation code (6.2 LOC per constraint), without introducing auxiliary validator classes. This result reflects that essential invariants are encoded declaratively via CAP annotations and automatically reconstructed into executable OCL enforcement logic.

Original DCSL provides partial support for essential constraints but requires handwritten methods for aggregation and dependency-based invariants, increasing manual LOC and auxiliary artifacts. In Apache Causeway and OpenXava, essential structural constraints beyond simple attribute validation are typically implemented imperatively, resulting in substantially higher manual LOC and additional validator/service classes. Actifsource provides model-level rule configuration; however, essential cross-association and aggregation constraints frequently require additional rule artifacts or configuration connectors.

Interpretation. The results indicate that integrating CAP into DCSL significantly reduces the handwritten effort required to enforce essential OCL constraints at the domain-model level. The reduction reflects per-constraint enforcement effort rather than relocation of equivalent logic, since OCL semantics are defined once in reusable CAP templates and not re-implemented for each constraint instance. All measurements are grounded in controlled reconstruction experiments using identical essential constraint sets across platforms.

6.5. RQ2: CAP Applicability to Domain Models

This subsection evaluates the applicability of CAP by measuring its coverage over constraints identified in three domain models: CourseMan, OrderMan, and ProcessMan. Constraints were classified into fourteen groups and mapped to the corresponding CAP patterns (or marked as out of scope), with classifications independently reviewed by two researchers. Table 5 reports group-level coverage and overall totals, illustrating how constraints across categories are captured by CAP patterns.

Table 5: Constraint groups and CAP patterns

Constraint Group	**CAP Catalog**	**Applicability of CAP**		
		CourseMan	OrderMan	Process-Man
Well-formedness constraints	DCSL in Table 1	11/11	11/11	11/11
Quantitative limits	SumConstraint	19/19	4/4	7/7
Dependency and prerequisite constraints	PrerequisiteConstraint	10/10	12/12	13/13
Scheduling and conflict constraints	ScheduleConstraint	17/17	6/6	10/10
Entity qualification constraints	EligibilityConstraint	20/20	5/5	12/12
Retry or reattempt rules	RetakeConstraint	8/8	2/2	8/8
Capacity constraints	SizeConstraint	17/17	3/3	15/15
Time and deadline constraints	TimeConstraint	10/10	6/6	13/13
Computation and derivation rules	SumProduct	15/15	8/8	8/8
Status-based constraints	StatusConstraint	12/12	6/6	15/15
Organizational structure constraints	StructuralConstraint	29/29	6/6	16/16
Pre-/post-condition constraints	✘	0/4	0/4	0/4
Access control constraints	✘	0/4	0/2	0/4
Automated reaction constraints	✘	0/4	0/2	0/4
	Total	**172/184**	**66/76**	**128/140**

Coverage metric. Applicability is computed as:

$$\text{Coverage Rate} = \frac{\text{Number of constraints representable by CAP}}{\text{Total identified constraints}}$$

Results. Across the three systems, CAP covers 11 out of 14 constraint groups and achieves high system-level coverage: 93.5% for CourseMan (172/184), 86.8% for OrderMan (66/76), and 91.4% for ProcessMan (128/140). The uncovered constraints belong to three groups—pre-/post-conditions, access control, and automated reactions—which require operational/event-driven semantics rather than reusable invariant templates. This scope definition preserves catalog conciseness while targeting reusable, model-level constraint families. Appendix C provides representative OCL-to-CAP mappings for each system.

Feasibility. Beyond coverage, feasibility is validated through the generation and execution of executable prototypes from the unified domain model with CAP, confirming that the approach can be realized as runnable software artifacts.

6.6. Discussion

We summarize the main implications of the evaluation.

Expressiveness. CAP captures recurring constraint families observed across three heterogeneous domains using annotation-based templates, enabling model-level specification and enforcement with minimal manual coding for the partially supported cases.

Required coding effort. Measured coding-effort indicators (Table 4 and Appendix B) show that CAP reduces handwritten validation logic and auxiliary validators by generating enforcement logic from reusable templates integrated with the domain model.

Effectiveness and maintainability. By consolidating constraints within a unified domain model and mapping them systematically to executable enforcement, CAP improves traceability and reduces duplicated/scattered rule implementations, thereby lowering maintenance overhead as the model evolves.

Scope boundaries. Constraint forms requiring explicit operational semantics (pre- and postcondition), security policy semantics (access control), or event-triggered execution ordering (automated reactions) are out of scope of the current CAP catalog. These categories are architecture-dependent and less reusable as invariant templates, and thus constitute natural extension points for future work.

7. Threats to Validity

This section outlines potential threats to the validity of both our proposed method and its evaluation. Following the classification by Runeson et al. [33], we organize these threats into three groups: construct validity, internal validity, and external validity.

7.1. Construct Validity

In our case study, we assumed that the domain requirements were fully captured without misinterpretation that could result in an unsatisfactory model or inconsistent business rules. The modeler collected all domain requirements to ensure that the constructed models were accurate and satisfactory. Our method mitigated the threat of misinterpretation by enabling the explicit representation of domain class models, OCL constraints, and parameterized specification template annotations. These were then encapsulated into a CAP pattern, which facilitated their composition into an executable unified domain model within a domain-driven architecture.

7.2. Internal Validity

One threat arises from translating high-level domain specifications (UML class diagrams, OCL constraints, and annotations) into a CAP template. To address this, we provided guidelines in Section 4 for applying CAPs with the initial catalog. Another threat is that OCL constraints may not capture all input patterns. We mitigated this by evaluating three complex applications (CourseMan, OrderMan and ProcessMan) and by planning to extend the catalog with additional patterns. Finally, implementation errors may occur when composing CAP concerns into a unified domain model and applying the JDA framework. To reduce this risk, we thoroughly tested and reviewed the code and released it publicly for community validation.

7.3. External Validity

The selection of three heterogeneous case studies (CourseMan, OrderMan, and ProcessMan), covering varying constraint densities and domain characteristics, mitigates this threat by demonstrating cross-domain applicability. Threats to external validity primarily concern the generalizability of the method to other DDD-based systems. First, as the approach targets AST-oriented modeling within DDD, its applicability may be limited in alternative architectural paradigms. Second, the representativeness of the selected case studies may be questioned. However, this risk is alleviated by the pattern-based design of CAP, which adheres to the well-established principle of keeping patterns generic and domain-independent.

8. Related Work

Several studies have focused on transforming class diagrams and business rules specified using OCL into executable code or annotations in OOPLs. For example, Cabot et al. [34] proposed a method for generating Java code from OCL constraints by translating them into SQL queries; however, their approach primarily targets database validation rather than in-memory domain model constraints. Similarly, Rackov et al. [35] explored Java code generation based on OCL rules, although their approach generates standalone validation methods rather than using annotations to embed constraints directly within the model.

Hutchison et al. [36] and Chimiak-Opoka et al. [37] proposed frameworks for translating OCL into Java annotations, but their solutions require significant manual intervention when handling complex constraints. OCL2Java, a semi-automated tool, mainly supports simple OCL patterns such as `and`, `or`, and `not`, but lacks support for more complex constructs such as `exists`, `forAll`, and set operations. More recently, several studies [12, 14–16] have explored the use of OCL combined with annotations to represent domain models in a DDD context. Dania et al. [38] developed the OCL2MSFOL approach, which translates OCL into first-order logic for formal verification but does not address implementation within domain models.

However, these approaches exhibit several limitations. First, they lack full automation, as most require manual intervention or user refinements when dealing with complex constraints. Second, they provide limited pattern support, often handling only simple model patterns while struggling with more complex OCL constructs such as `exists`, `forAll`, or set operations on classes, attributes, associations, and operations [37]. Third, accuracy and efficiency issues arise in some approaches, where generated annotations do not fully preserve the semantics of the original OCL constraints, particularly for quantifiers and collection operations, or where inefficient validation code leads to performance bottlenecks in large-scale systems. Fourth, integration challenges remain, as most solutions focus solely on constraint validation without addressing the derivation of calculated values or the propagation of changes within the domain model, creating a disconnect between validation logic and the natural evolution of the model state. Finally, tooling complexity is a concern, as many frameworks require separate code generation steps and specialized tools, increasing development overhead and maintenance effort when models evolve.

Our previous work [20] introduced the DCSL as an annotation-based internal DSL for specifying structural aspects of domain classes in DDD. That work primarily focused on representing essential structural constraints—such as identity, mutability, cardinality bounds, and value ranges—and enabling automatic generation of executable domain models from annotated class definitions.

The present work extends this foundation in several fundamental ways. First, the original DCSL provided a fixed set of predefined structural constraints directly mapped to annotation attributes. In contrast, the current approach introduces Constraint Annotation Patterns (CAPs) as reusable and parameterized templates that abstract families of OCL invariants. This shifts the language from supporting isolated structural properties to supporting structured abstraction of complex constraint semantics. Second, while the original DCSL handled essential invariants at the class and attribute level, it did not provide a systematic mechanism for modeling higher-level constraint structures such as aggregation constraints, dependency rules, scheduling constraints, or status-based invariants. The CAP-based extension introduces a pattern catalog that generalizes and modularizes these constraint types. Third, the current work introduces explicit concern-oriented composition mechanisms. In earlier DCSL work, annotations were attached directly to domain classes without a formalized composition layer. In this paper, CAPs are organized, managed, and composed into a Unified Domain Model (UDM), enabling structured integration of multiple constraint concerns. Fourth, the present contribution formalizes the generation mapping between CAP instances and OCL invariants, providing a clearer semantic correspondence between abstract constraint patterns and executable specifications. This formalization was not part of the original DCSL proposal.

Consequently, the present work represents a conceptual and architectural generalization of DCSL. Rather than introducing additional annotations, it elevates constraint specification to a reusable pattern layer with formal generation semantics and concern-oriented composition, thereby redefining the role of DCSL within a unified domain modeling framework. Annotation in annotation-based patterns enhances clarity and contextual understanding, improving collaboration, traceability, and domain alignment in complex adaptive systems.

9. Conclusion and Future Work

This paper presented Constraint Annotation Patterns (CAP), a pattern-based extension to annotation-driven DSL for representing complex OCL invariants within DDD. By introducing reusable and parameterized constraint templates, CAP bridges formal constraint semantics and executable domain models while preserving alignment between model and implementation.

The proposed approach enhances model-level expressiveness without redefining OCL semantics. Empirical evaluation across three heterogeneous case studies demonstrates that CAP reduces repetitive validation code, improves maintainability through pattern reuse, and achieves high coverage of invariant-based constraint families.

CAP does not eliminate formal modeling effort; instead, it consolidates it into a reusable abstraction layer that amortizes complexity across projects. This design supports long-term scalability and systematic evolution of domain models.

Future work includes expanding the CAP catalog toward additional constraint families, investigating extensions for operational and event-driven semantics, and evaluating performance characteristics in large-scale industrial systems. We also plan to explore cross-platform generation strategies to support multiple host programming languages.

We also plan to explore intelligent assistance mechanisms (e.g., LLM-supported pattern suggestion) for the semi-automated discovery of new CAP families. In particular, large language models could help identify recurring invariant patterns and suggest candidate parameterized templates, while preserving formal validation and expert oversight.

CRediT authorship contribution statement

Van-Vinh Le: Conceptualization, Methodology, Writing- Original draft preparation, Investigation, practical in real-world DDD projects, Validation, Writing- Reviewing and Editing. Duc-Hanh Dang: Conceptualization, Methodology, Formal analysis, Validation, Writing- Reviewing and Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2004.

[2] OMG, *Object Constraint Language 2.4*. OMG, 2014. [Online]. https://www.omg.org/spec/OCL/2.4/PDF

[3] V. Vernon, *Implementing domain-driven design*, 1st ed. Addison-Wesley Professional, 2013.

[4] O. Özkan, Babur, and M.v.d. Brand, "Domain-Driven Design in Software Development: A Systematic Literature Review on Implementation, Challenges, and Effectiveness," Nov. 2023. [Online]. http://arxiv.org/abs/2310.01905

[5] S.K. Jaiswal and R. Agrawal, "Domain-Driven Design (DDD)- Bridging the Gap between Business Requirements and Object-Oriented Modeling," *Int. Innovative Research in Engineering and Management*, Vol. 11, No. 2, 2024, pp. 79–83, number: 2.

[6] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*, 2nd ed. Morgan&Claypool, 2017.

[7] J. Griffin, "Domain-Driven Laravel, Learn to Implement Domain-Driven Design Using Laravel," Jan. 2021.

[8] OMG, *Unified Modeling Language 2.5.1*. Object Management Group, 2017.

[9] M. Fowler, *Domain-Specific Languages*, 1st ed. Addison-Wesley Professional, Sep. 2010.

[10] Andrzej Wąsowski and Thorsten Berger, *Domain-Specific Languages Effective Modeling, Automation, and Reuse*. Springer Cham, 2023.

[11] M. Völter, S. Benz, C. Dietrich, B. Engelmann, M. Helander et al., *DSL Engineering - Designing, implementing and using domain-specific languages*. Stuttgart, Germany: M Volter / DSLBook.org, 2013. [Online]. http://www.dslbook.org/

[12] I. Córdoba-Sánchez and J. De Lara, "Ann: A domain-specific language for the effective design and validation of Java annotations," *Computer Languages, Systems Structures*, Vol. 45, Apr. 2016, pp. 164–190.

[13] F.U. Haq and J. Cabot, "B-OCL: An Object Constraint Language Interpreter in Python," Mar. 2025, arXiv:2503.00944 [cs]. [Online]. http://arxiv.org/abs/2503.00944

[14] C. Noguera and L. Duchien, "Annotation Framework Validation Using Domain Models," in *Model Driven Architecture – Foundations and Applications*. Springer, Berlin, Heidelberg, 2008, pp. 48–62, iSSN: 1611-3349. [Online]. https://link.springer.com/chapter/10.1007/978-3-540-69100-6_4

[15] A.D. Brucker, M.P. Krieger, D. Longuet, and B. Wolff, "A Specification-Based Test Case Generation Method for UML/OCL," in *Models in Software Engineering*. Springer, Berlin, Heidelberg, 2011, pp. 334–348, iSSN: 1611-3349. [Online]. https://link.springer.com/chapter/10.1007/978-3-642-21210-9_33

[16] S. Ali, M. Zohaib Iqbal, A. Arcuri, and L.C. Briand, "Generating Test Data from OCL Constraints with Search Techniques," *IEEE Transactions on Software Engineering*, Vol. 39, No. 10, Oct. 2013, pp. 1376–1402, conference Name: IEEE Transactions on Software Engineering. [Online]. https://ieeexplore.ieee.org/document/6491405

[17] J.G.B.J.G. Steele and G.B.A. Buckley, *The Java language Specificition*. Bhaskarjyoti Saikia, 2020.

[18] Dan Haywood, "Apache Isis - Developing Domain-Driven Java Apps," *Methods & Tools: Practical knowledge source for software development professionals*, Vol. 21, No. 2, 2013, pp. 40–59.

[19] J. Paniza, *Learn OpenXava by example*, 1st ed. CreateSpace, 2011.

[20] D.M. Le, D.H. Dang, and V.H. Nguyen, "On domain driven design using annotation-based domain specific language," *Computer Languages, Systems & Structures*, Vol. 54, Dec. 2018, pp. 199–235.

[21] D.H. Dang, D.M. Le, and V.V. Le, "AGL: Incorporating behavioral aspects into domain-driven design," *Information and Software Technology*, Vol. 163, Nov. 2023, p. 107284.

[22] V.V. Le and D.H. Dang, "An Approach to Composing Concerns for an Executable Unified Domain Model," in *2024 RIVF Int. Conf. On Computing and Communication Technologies*

(RIVF), Dec. 2024, pp. 424–428, iSSN: 2473-0130. [Online]. https://ieeexplore.ieee.org/abstract/document/11009109

[23] J. Sangabriel-Alarcón, J.O. Ocharán-Hernández, K. Cortés-Verdín, and X. Limón, "Domain-Driven Design for Microservices Architecture Systems Development: A Systematic Mapping Study." IEEE Computer Society, Nov. 2023, pp. 25–34. [Online]. https://www.computer.org/csdl/proceedings-article/conisoft/2023/288300a025/1Y7jTxtUNBm

[24] V.V. Le, N.T. Be, and D.H. Dang, "On Automatic Generation of Executable Domain Models for Domain-Driven Design," in *2023 15th International Conference on Knowledge and Systems Engineering (KSE)*, Oct. 2023, pp. 1–6, iSSN: 2694-4804. [Online]. https://ieeexplore.ieee.org/document/10299453

[25] A. Wąsowski and T. Berger, "Internal Domain-Specific Languages," in *Domain-Specific Languages: Effective Modeling, Automation, and Reuse*. Springer, Cham, 2023, pp. 357–394.

[26] B. Liskov and J. Guttag, *Program development in Java: abstraction, specification, and object-oriented design*. Addison-Wesley Professional, 2000.

[27] J.A. Hoffer, J. George, and J.A. Valacich, *Modern Systems Analysis and Design*, 7th ed. Pearson, 2013.

[28] V.V. Le and D.H. Dang, "Constraint Annotation Patterns for Prototype Generation within Domain-Driven Design," Tech. Rep., 2025. [Online]. https://tinyurl.com/techrepor

[29] D.M. Le, D.H. Dang, and H.T. Vu, "jDomainApp: A Module-Based Domain-Driven Software Framework," in *Proc. 10th Int. Symposium on Information and Communication Technology (SoICT)*, Dec. 2019, pp. 399–406.

[30] F. Büttner and M. Gogolla, "On ocl-based imperative model transformation," *Electronic Communications of the EASST*, Vol. 29, 2010.

[31] Thakur and U. Pandey, "The Role of Model-View Controller in Object Oriented Software Development," *Nepal Journal of Multidisciplinary Research*, Vol. 2, Nov. 2019, pp. 1–6.

[32] *Actifsource*. Actifsource AG, Switzerland - all rights reserved., 2017. [Online]. https://www.actifsource.com/_downloads/ActifsourceManual_ActifsourceUserManual.pdf

[33] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, Vol. 14, No. 2, Apr. 2009, pp. 131–164.

[34] J. Cabot and E. Teniente, "Transformation techniques for OCL constraints," *Science of Computer Programming*, Vol. 68, No. 3, Oct. 2007, pp. 179–195. [Online]. https://www.sciencedirect.com/science/article/pii/S0167642307001256

[35] M. Rackov, S. Kaplar, M. Filipović, and G. Milosavljević, "Java code generation based on ocl rules," in *6th International Conference on Information Society and Technology*, 2016, pp. 191–196.

[36] L. Hamann, O. Hofrichter, and M. Gogolla, "OCL-Based Runtime Monitoring of Applications with Protocol State Machines," in *Modelling Foundations and Applications*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, Vol. 7349, pp. 384–399, series Title: Lecture Notes in Computer Science. [Online]. http://link.springer.com/10.1007/978-3-642-31491-9_29

[37] J. Chimiak-Opoka, B. Demuth, A. Awenius, D. Chiorean, S. Gabel et al., "OCL tools report based on the ide4OCL feature model," *Electronic Communications of the EASST*, Vol. 44, 2011. [Online]. https://eceasst.org/index.php/eceasst/article/view/1850

[38] C. Dania and M. Clavel, "OCL2MSFOL: a mapping to many-sorted first-order logic for efficiently checking the satisfiability of OCL constraints," in *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems*. Saint-malo France: ACM, Oct. 2016, pp. 65–75. [Online]. https://dl.acm.org/doi/10.1145/2976767.2976774

A. Expressiveness Evaluation Protocol and Detailed Results

This appendix provides the detailed protocol and supporting evidence for the expressiveness results reported in Section 6.3. It complements Table 3 by (i) defining the expressiveness criteria, (ii) clarifying the evaluation procedure and measurement interpretation, and (iii) documenting how each platform supports each criterion through built-in constructs or requires manual implementation.

A.1. Evaluation Protocol

We followed the controlled reconstruction protocol described in Section 6.2. For each case study (CourseMan, ProcessMan, and OrderMan), we first identified recurring semantic OCL constraint types (constraint families) derived from the UML/OCL specifications and domain requirements. Each constraint family corresponds to one expressiveness criterion.

Selection of representative constraints. For each criterion, we selected representative constraints from each case study whenever available. Concretely, we mapped each criterion to representative OCL invariants in the case study models and evaluated whether each approach can express these rules declaratively.

- If a constraint family appears in multiple case studies, we reconstructed at least one representative constraint per case;
- When multiple variants exist, we selected the structurally most demanding variant (e.g., deeper navigation chains, nested quantifiers, or conditional logic);
- All ratings are grounded in actual reconstruction artifacts rather than feature lists or documentation claims.

What counts as manual validation code. In this evaluation, *manual validation code* includes any handwritten imperative logic required to enforce a constraint, such as:

- custom validation methods,
- framework-specific validator/service classes,
- event hooks or callback handlers,
- glue code required to bridge model annotations and runtime enforcement.

Generated code derived automatically from model annotations or meta-model specifications is not counted as manual code.

Rating rubric. For each expressiveness criterion, we assign one of three levels based on reconstruction artifacts and measurable thresholds:

- Fully supported, denoted by "$++$": expressible and enforceable using native annotations/constructs, with no additional manual validation code;
- Partially supported, denoted by "$+$" : enforceable with minor custom code, bounded by ≤ 20 LOC per representative constraint;
- Not supported, denoted by "$-$": requires manual implementation with > 20 LOC per constraint or scattered imperative enforcement logic.

The assessment was performed independently by two researchers. Disagreements were resolved by re-checking the encoding artifacts and reaching consensus.

A.2. Expressiveness Criteria

The twelve criteria correspond to constraint families that recur across the three case studies. They focus strictly on *constraint expressiveness*, i.e., the types of domain integrity rules that can be captured declaratively, rather than on general tool or framework features.

1. **Cardinality and multiplicity bounds**: constraints that restrict the size of collections or enforce multiplicity limits on associations (e.g., minimum/maximum number of related objects).
2. **Value range constraints**: constraints restricting the allowable values of attributes, including numeric intervals, upper/lower bounds, and string length limits.
3. **Uniqueness and key constraints**: rules ensuring that attribute values or combinations of attributes remain unique across instances of a class.
4. **Aggregation constraints**: constraints involving aggregation over collections, such as `sum` or `count`, often used to enforce thresholds or reconciliation conditions.
5. **Conditional invariants**: constraints whose validity depends on a condition, typically expressed in an if–then form based on attribute values or object state.
6. **Cross-entity dependency constraints**: rules expressing dependencies across associated entities, such as eligibility or prerequisite requirements spanning multiple classes.
7. **Scheduling and conflict constraints**: constraints preventing conflicting allocations, such as overlapping schedules, concurrent assignments, or resource conflicts.
8. **Time and deadline constraints**: rules involving temporal ordering, deadlines, or validity windows between events or timestamps.
9. **Status-based constraints**: constraints that depend on the lifecycle state or status attribute of an entity (e.g., rules activated only in specific states).
10. **Derived/computed value consistency**: constraints ensuring consistency between derived values and their underlying attributes (e.g., weighted values, proportional thresholds, or computed attributes).
11. **Quantification over collections**: constraints expressed using universal or existential quantification over object collections (e.g., `forAll`, `exists` patterns).
12. **Constraint composition support**: the ability to combine multiple constraints declaratively within the model, avoiding scattered manual validation code.

A.3. Evidence Table

Table 6 reports detailed evidence per criterion. For each platform, we indicate the primary construct used and the resulting rating according to the rubric above.

Table 6: Detailed expressiveness evidence for CAP, DCSL, OpenXava, Apache Causeway, and Actifsource

Criterion	**CAP**	**DCSL**	**OpenXava**	**Apache Causeway**	**Actifsource**
Cardinality / multiplicity bounds	(++) `@SizeConstraint` (collection size bounds)	(+) `@Cardinality`	(+) collection annotations + custom checks	(+) property annotations + custom checks	(++) multiplicity in meta-model
Value range constraints	(++) `@DAttr(min,max,len)` + CAP variants	(+) `@DAttr(min,max,len)`	(+) bean validation (partial)	(+) property constraints	(++) meta-model constraints
Uniqueness / key constraints	(++) `@DAttr(unique=true)` + CAP support	(+) `@DAttr(unique=true)` (limited)	(+) partial (often DB-level)	(+) partial/-manual	(++) meta-model uniqueness
Aggregation constraints	(++) `@SumConstraint` (sum/count variants)	(−) not supported	(−) manual implementation	(−) manual implementation	(+) model rules/generators
Conditional invariants	(++) parameterized CAP templates	(−) not supported	(+) method-based checks	(+) custom imperative logic	(+) model-level rules
Cross-entity dependency constraints	(++) `@PrerequisiteConstraint` `@EligibilityConstraint`	(+) partial structural support	(−) manual implementation	(+) custom logic	(+) model rules
Scheduling and conflict constraints	(++) `@ScheduleConstraint`	(−) not supported	(−) manual implementation	(−) manual implementation	(+) model rules
Time and deadline constraints	(++) `@TimeConstraint`	(−) not supported	(+) partial (bean validation)	(+) partial property checks	(++) model-level rules
Status-based constraints	(++) `@StatusConstraint`	(−) not supported	(−) manual implementation	(+) custom logic	(+) model rules
Derived/computed value consistency	(++) `@SumProduct` (derived computation templates)	(+) limited derived support	(+) built-in calculation (partial)	(+) custom property logic	(++) derived attributes in meta-model
Quantification over collections (`forAll`/`exists`)	(++) CAP quantification templates	(−) not supported	(−) manual implementation	(−) manual implementation	(+) model-level rules
Constraint composition support	(++) CAP catalog composition	(+) partial structural combination	(−) scattered custom code	(−) scattered custom code	(+) meta-model configuration

Provenance. All ratings are derived from reconstruction artifacts of CourseMan, ProcessMan, and OrderMan. Representative OCL-to-CAP mappings are documented in the case-specific appendices.

B. Required Coding Effort: Measurement Protocol and Evidence

This appendix provides the detailed measurement protocol and supporting evidence for the required coding effort results reported in Section 6.4. All values were obtained from the controlled reconstruction experiments described in Section 6.2 and are grounded in the actual implementation artifacts of CourseMan, ProcessMan, and OrderMan.

B.1. Operational Definition of Coding Effort

Required coding effort evaluates the amount of handwritten imperative logic necessary to obtain executable constraint enforcement beyond structural model definitions. In this appendix:

- **Manual validation code** includes any handwritten imperative code required to enforce constraints, such as validation methods, service classes, custom validators, event hooks, or glue code.
- **Generated code** refers exclusively to code automatically produced from model annotations or meta-model configurations.
- Structural class definitions and attribute declarations are not counted as validation code.

B.2. Measured Indicators

For each tool and case study, we define the measures as follows:

1. **Number of manual validation methods** required to encode constraints;
2. **Manual validation LOC per constraint**, computed as:

$$\text{Average LOC} = \frac{\text{Total handwritten validation LOC}}{\text{Number of constraints requiring manual code}}$$

3. **Automation percentage**, defined as:

$$\text{Automation } \% = \frac{\text{Number of constraints enforced without manual validation code}}{\text{Total number of constraints}} \times 100$$

4. **Additional validator/service classes** introduced solely for constraint enforcement.

All LOC counts exclude comments and blank lines. Measurements were collected directly from reconstructed source artifacts.

B.3. Rating Rubric

The categorical ratings reported in Table 7 were derived using the following thresholds:

- Full automation, denoted by "$++$": more than 80% of constraint enforcement logic generated; average manual validation code $\leq$ 10 LOC per constraint; no additional validator/service classes required.
- Partial automation, denoted by "$+$": 30–80% generated enforcement logic; average manual validation code between 10 and 50 LOC per constraint; additional validators/services required for complex cases.
- Manual implementation, denoted by "$-$": less than 30% generated enforcement logic or average manual validation code $>$ 50 LOC per constraint, indicating predominantly imperative implementations.

  These thresholds were applied uniformly across all three case studies.

B.4. Learning Curve Measurement

Learning curve was assessed using structural onboarding complexity indicators rather than subjective perception. For each platform, we recorded:

1. Number of core DSL constructs/annotations required to model constraints;
2. Number of configuration steps before obtaining the first executable prototype;
3. Whether external meta-model configuration or IDE-specific setup was required.

Rating levels were defined as:

- Low onboarding complexity, denoted by "$++$": ≤ 5 new constructs, ≤ 3 configuration steps, no external meta-model setup.
- Moderate complexity, denoted by "$+$": 6–10 constructs, 3–6 configuration steps, limited framework configuration.
- High complexity, denoted by "$-$": > 10 constructs, > 6 setup steps, meta-model or IDE-specific configuration required.

Table 7: Comparison of platforms based on required coding effort (categorical summary)

Tool\Criterion	**CAP-extended DCSL**	**Apache**	**OpenXava**	**Actifsource**
Automation of code generation	++	+	+	+
Support for complex constraints	++	-	+	+
Integration with domain model	++	+	+	+
Learning curve	++	+	+	-
Flexibility and customization	+	+	+	++

Table 7 summarizes the five coding-effort criteria derived from the measurements in this subsection. The ratings $(++, +, -)$ denote low, moderate, and high implementation complexity. CAP-extended DCSL achieves the highest automation and lowest onboarding complexity, whereas alternative platforms require additional imperative validation logic or meta-model configuration.

B.5. Tool-Specific Evidence

This subsection provides qualitative interpretation grounded in the quantitative measurements above.

Table 8: Required coding level evidence for CAP-extended DCSL

Criterion	**Evidence**
Automation of code generation	92% automation; average 6.2 LOC per constraint; no additional validator classes.
Support for complex constraints	Aggregation, dependency, scheduling, temporal, and status constraints encoded using reusable CAP templates without imperative logic.
Integration with domain model	Constraints declared directly via annotations within domain classes; no external rule files.
Learning curve	≤ 5 core CAP annotations; ≤ 3 setup steps; no meta-model configuration required.
Flexibility and customization	Extension through additional pattern templates within the CAP catalog.

Table 8 summarizes qualitative evidence for the required coding level of CAP-extended DCSL. Constraints are specified declaratively via CAP annotations embedded in domain classes, enabling high automation and minimal manual validation logic.

Table 9: Required coding level evidence for Apache Causeway

Criterion	**Evidence**
Automation of code generation	22% automation; average 54.3 LOC per constraint; 6 auxiliary validator/service classes.
Support for complex constraints	Most non-trivial constraints require imperative Java methods.
Integration with domain model	Annotation-driven UI and persistence; constraint semantics externalized in methods.
Learning curve	Moderate onboarding; framework conventions required.
Flexibility and customization	Custom domain services and validation methods extensible.

Table 8 summarizes qualitative evidence for the required coding level of CAP-extended DCSL. Constraints are specified declaratively via CAP annotations embedded in domain classes, enabling high automation and minimal manual validation logic.

Table 10: Required coding level evidence for OpenXava

Criterion	**Evidence**
Automation of code generation	45% automation; average 37.5 LOC per constraint; 4 auxiliary validator classes.
Support for complex constraints	Basic annotation support; complex business rules implemented imperatively.
Integration with domain model	Entity-centric modeling; validation logic often externalized.
Learning curve	Moderate onboarding; framework configuration required.
Flexibility and customization	Custom validators permitted; increased manual effort for complex rules.

Table 10 summarizes the evidence for OpenXava. Although basic validation annotations are available, complex domain constraints typically require custom validator classes, increasing implementation effort.

Table 11: Required coding level evidence for Actifsource

Criterion	**Evidence**
Automation of code generation	52% automation; average 31.8 LOC per constraint; 3 auxiliary rule/-configuration artifacts.
Support for complex constraints	Model-level rules supported; OCL-style invariants require configuration or rule scripting.
Integration with domain model	Strong meta-model integration; runtime enforcement requires connectors.
Learning curve	High onboarding complexity; meta-model and Eclipse configuration required.
Flexibility and customization	High extensibility at meta-model level.

Table 11 summarizes qualitative evidence for Actifsource. While the platform supports model-level rules, enforcing OCL-style invariants often requires additional configuration or scripting, resulting in higher onboarding complexity.

Provenance. All quantitative measurements originate from reconstruction artifacts of CourseMan, ProcessMan, and OrderMan. LOC counts and automation ratios were computed directly from the implemented validation code for each platform.

C. Case-level Mapping Evidence for CourseMan, ProcessMan, and OrderMan

This appendix provides representative OCL invariants and their mappings to CAP pattern families for the three case studies. The goal is to make the aggregated coverage results in Section 6.5 and the expressiveness analysis in Appendix A traceable to concrete encoding artifacts.

CourseMan: Representative OCL-to-CAP Mappings

Table 12: Representative OCL invariants in CourseMan and mapped CAP families

OCL invariant (excerpt)	Constraint intent (summary)	CAP
`Student inv TotalTuitionEqualsSum: sum(tuitionFee)=tuitionTotal`	Reconciliation of tuition total for current semester	`SumConstraint`
`Course inv NoCyclicDependency: closure(...)->excludes(self)`	Acyclic prerequisite dependency	`PrerequisiteConstraint`
`Student inv NoTimeConflict: schedule intersection isEmpty()`	No overlap in enrolled classes	`ScheduleConstraint`
`Enrolment inv ThesisEligibility: completedCredits >= 0.8*totalCredits`	Eligibility threshold for specialization/capstone	`EligibilityConstraint`
`SClass inv MaxEnrollment: enrolments->size() <= maxStudents`	Capacity constraint	`SizeConstraint`
`Student inv PaymentDeadline: feePaid or deadline > now()+21`	Flag-or-deadline (lead-time) rule	`TimeConstraint`
`Student::GPA derive: sum(gradePoints*credits)/sum(credits)`	Weighted computation rule	`SumProduct`
`Student inv AcademicSuspension: warnings->size()>2 implies status='Suspended'`	Status-triggered restriction	`StatusConstraint`
`Student inv UniqueEnrolmentPerModule: no duplicate course enrolment`	Collection consistency	`StructuralConstraint`

Table 12 presents representative OCL invariants from the CourseMan case study and their corresponding CAP mappings. The examples illustrate common academic constraints such as credit limits, enrollment restrictions, and prerequisite conditions. These invariants are systematically captured using reusable CAP templates, demonstrating how recurring domain constraints can be abstracted into pattern-based specifications.

ProcessMan: Representative OCL-to-CAP Mappings

Table 13: Representative OCL invariants in ProcessMan and mapped CAP families

OCL invariant (excerpt)	Constraint intent (summary)	CAP
`Process inv TotalPlannedEqualsTasks: totalPlannedHours = sum(task.plannedHours)`	Aggregation reconciliation (plan equals sum of tasks)	`SumConstraint`

OCL invariant (excerpt)	Constraint intent (summary)	CAP
`SubjectBySemester inv TotalHoursWithinBounds: sum(assignedHours) in [min,max]`	Aggregated bounds over assignments	`SumConstraint`
`Process inv ActiveEnforcesEquality: if active then sum=plan else sum<=plan`	Conditional enforcement over aggregation	`SumConstraint`

Table 13 shows representative OCL invariants from the ProcessMan case study and their mapping to CAP templates. The examples illustrate aggregation constraints over process tasks, captured using reusable templates such as `SumConstraint`.

OrderMan: Representative OCL-to-CAP Mappings

Table 14: Representative OCL invariants in OrderMan and mapped CAP families

OCL invariant (excerpt)	Constraint intent (summary)	CAP
`Order inv TotalAmountEqualsSum: sum(unitPrice*qty)=totalAmount`	Reconciliation of order total over current-period billable lines	`SumConstraint`
`Order inv AmountWithinLimit: sum(unitPrice*qty) <= maxAmountLimit`	Financial ceiling constraint at order level	`SumConstraint`
`Order inv MinUnitsUnlessExempt: not exempt implies sum(qty) >= 5`	Conditional minimum requirement with exemption flag	`SumConstraint`
`Order inv OnHoldPremiumLineLimit: ONHOLD implies count(premiumLines) <= 2`	Conditional count cap under ONHOLD state	`SumConstraint`

Table 14 presents representative OCL invariants from the OrderMan case study and their mapping to CAP templates. The examples illustrate financial and quantity-related aggregation constraints in order processing, also captured using `SumConstraint`.

Out-of-scope Constraint Forms

Across the three systems, we also identified constraint forms that are not currently targeted by the CAP catalog because they depend on operational or deployment-specific semantics rather than reusable invariant templates: (i) pre-and postconditions requiring explicit pre-state and post-state reasoning, (ii) access control constraints dependent on platform-specific authorization mechanisms, and (iii) automated reaction constraints that couple events to actions and require execution ordering. These forms are reported explicitly as uncovered in Table 5.

Authors and affiliations

Van-Vinh Le
e-mail: levanvinh@vuted.edu.vn
ORCID: https://orcid.org/0000-0003-3626-7447
(1)Faculty of Information Technology, VNU University of Engineering and Technology,
(2) Vietnam National University, Hanoi
(3) Vinh University of Technology Education

Duc-Hanh Dang
e-mail: hanhdd@vnu.edu.vn
ORCID: https://orcid.org/0000-0003-4564-4080
(1) Faculty of Information Technology, VNU University of Engineering and Technology,
(2) Vietnam National University, Hanoi